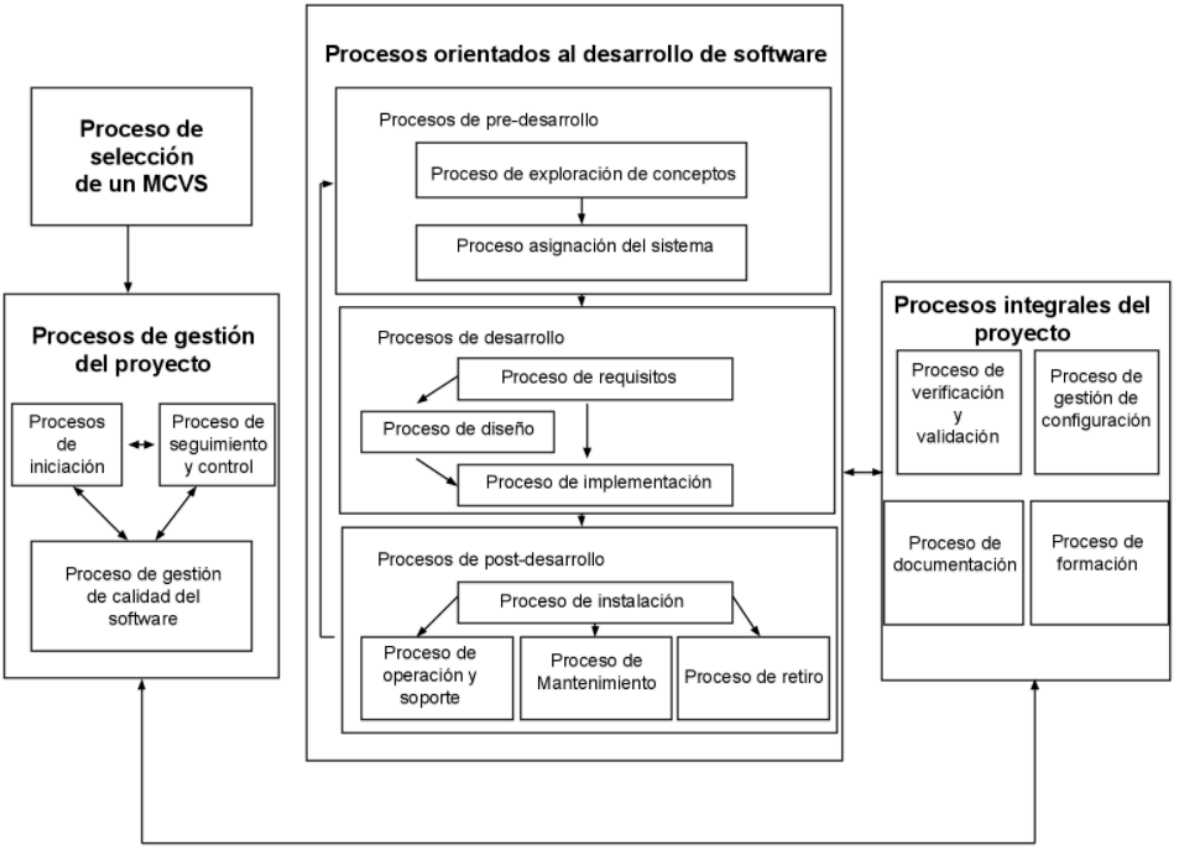
Riesgos

Introducción -Repaso IEEE 107



Riesgos:

- Tienen relación con el futuro
- Implican cambios
- Conllevan incertidumbre

Estrategias

Reactiva

Proactiva

1. Definición y Pilares del Riesgo (Según Charette)

La definición de riesgo se basa en tres pilares conceptuales clave:

- Riesgo Futuro: Afecta a **acontecimientos futuros** (lo que podría ocurrir). La gestión del riesgo busca cambiar las acciones actuales para crear una oportunidad mejor en el futuro.
- Implica Cambio: El riesgo siempre conlleva la posibilidad de **cambio** (en opiniones, acciones, tecnología, requisitos, etc.).
- Implica Elección: El riesgo está ligado a la **elección** y la incertidumbre inherente a tomar una decisión (ej., qué métodos usar, cuánto énfasis dar a la calidad).

2. Estrategias de Riesgo: Reactivas vs. Proactivas

Estrategia	Descripción	Enfoque
Reactiva	"Escuela de Indiana Jones" o "Método de bomberos." Espera a que los problemas ocurran. En el mejor de los casos, se reservan recursos por si acaso; lo más común es corregir el problema apresuradamente una vez que ya ha fallado. Lleva a la "gestión de crisis."	Corregir (después de que ocurre).
Proactiva	Es la estrategia inteligente. Se inicia antes de los trabajos técnicos. Implica identificar, valorar (probabilidad/impacto) y priorizar los riesgos. El primer objetivo es evitar el riesgo; el segundo es tener un plan de contingencia para responder de manera controlada.	Prevenir y Planificar (antes de que ocurra).

3. Características y Categorías de los Riesgos del Software:

Características (Acuerdo Común):

El riesgo en software siempre implica:

- **Incertidumbre:** El evento puede o no puede ocurrir (no hay riesgos del 100% de probabilidad).
- **Pérdida:** Si el riesgo se materializa, habrá consecuencias no deseadas o pérdidas.

Categorías Principales (Por el Objeto Amenazado):

- **Riesgos del Proyecto:** Amenazan el plan (presupuesto, planificación temporal, personal, recursos, cliente, requisitos). Si se cumplen, el proyecto se retrasa y los costos aumentan.
- **Riesgos Técnicos:** Amenazan la calidad y la planificación temporal del software. Si se cumplen, la implementación puede ser difícil o imposible (problemas de diseño, verificación, ambigüedad en especificaciones, tecnologías anticuadas).
- **Riesgos del Negocio:** Amenazan la viabilidad del software a construir. Ponen en peligro el producto o el proyecto (ej., riesgo de mercado, riesgo estratégico, riesgo de dirección, riesgo de presupuesto).

Categorías Generales (Según Charette):

- **Riesgos Conocidos:** Identificables mediante una cuidadosa evaluación del plan, entorno técnico y comercial (ej., fechas de entrega no realistas).
- **Riesgos Predecibles:** Extrapolados de la **experiencia** en proyectos anteriores (ej., cambio de personal, mala comunicación con el cliente).
- **Riesgos Impredecibles:** El "comodín," extremadamente difícil de identificar por adelantado.

4. Identificación del Riesgo: La identificación de riesgos es un intento sistemático para especificar las amenazas al plan del proyecto.

- **Objetivo:** Identificar riesgos **conocidos y predecibles** para poder **evitarlos** o **controlarlos**.
- **Tipos de Riesgos a Identificar:**
    - **Riesgos Genéricos:** Amenazas potenciales para **todos** los proyectos de software.
    - **Riesgos Específicos del Producto:** Solo los pueden identificar aquellos con una **visión clara** del proyecto en cuestión (tecnología, personal, entorno).

Método de Lista de Comprobación (Checklist):

Una forma sistemática de identificar riesgos genéricos y predecibles es usando una Lista de Comprobación que se enfoca en las siguientes subcategorías:

1. **Tamaño del Producto:** Riesgos asociados con el tamaño total del software (LDC, PF, base de datos).
2. **Impacto en el Negocio:** Riesgos asociados con las limitaciones impuestas por la gestión o el mercado (ej., fechas límite, viabilidad del producto).
3. **Características del Cliente:** Riesgos asociados con la habilidad de comunicación y la sofisticación del cliente.

4. **Definición del Proceso:** Riesgos asociados con qué tan bien definido y seguido está el proceso de software.
5. **Entorno de Desarrollo:** Riesgos asociados con la disponibilidad y calidad de las herramientas.
6. **Tecnología a Construir:** Riesgos asociados con la complejidad y el uso de tecnología de punta.
7. **Tamaño y Experiencia de la Plantilla:** Riesgos asociados con la experiencia técnica del personal.

5. Riesgos Específicos del Checklist (Ejemplos)

Subcategoría	Riesgos Clave a Preguntar	Conclusión
Tamaño del Producto (6.3.1)	¿Tamaño estimado en LDC/FP? ¿Grado de seguridad de la estimación? ¿Número de cambios previstos en requisitos? ¿Tamaño de la base de datos?	Si hay una gran desviación respecto a la experiencia anterior o si los resultados anteriores fueron insatisfactorios, el riesgo es grande.
Impacto en el Negocio (6.3.2)	¿El efecto del producto en los ingresos? ¿Es razonable la fecha límite? ¿Consistencia de las necesidades del cliente? ¿Costos asociados por retraso o producto defectuoso?	Las consideraciones del negocio (marketing) pueden entrar en conflicto con la realidad técnica. Una gran desviación o resultados insatisfactorios anteriores indican gran riesgo.

Conceptos que ya tienes definidos o explicados: Prácticamente todos los términos clave de la gestión de riesgos que aparecen en el texto ya están cubiertos de forma clara en este resumen.

- **Riesgo / Riesgo de Software:** Definición y sus dos características (Incertidumbre y Pérdida).
- **Mitigación:** Es el objetivo principal de la estrategia proactiva (junto con la evitación y el plan de contingencia).

- **Tipos de Riesgos:** Riesgos de Proyecto, Técnicos, de Negocio, Conocidos, Predecibles e Impredecibles.

1. Riesgos de la Identificación Detallada (Checklist): Para identificar riesgos conocidos y predecibles, se utilizan listas de comprobación enfocadas en aspectos específicos del proyecto. Una respuesta negativa a la mayoría de las preguntas en estas listas indica un riesgo alto.

Riesgos Relacionados con el Cliente (6.3.3): Los clientes representan una amenaza significativa al plan del proyecto (presupuesto y planificación temporal), ya que sus características, personalidad y la calidad de la relación con el proveedor pueden crear un alto riesgo.

Factores de Riesgo del Cliente

Los clientes varían en varios aspectos que impactan el proyecto:

- **Necesidades y Conocimiento:** Algunos saben exactamente lo que quieren, otros no lo tienen claro; algunos se satisfacen con promesas vagas, otros exigen detalles.
- **Personalidad:** Algunos son negociadores entusiastas, otros preferirían no ser clientes. Hay quienes aceptan cualquier cosa y otros que se quejan de la falta de calidad.
- **Relación con el Proveedor:** Algunos tienen una relación sólida y conocida con el proveedor; otros se comunican de forma esporádica o impersonal.
- **Expectativas Contradictorias:** A menudo desean todo de inmediato y de forma gratuita, atrapando al producto en sus propias contradicciones.

Lista de Comprobación para Identificar Riesgos: Para evaluar si un cliente representa un riesgo significativo, se deben responder preguntas clave. Una respuesta "no" a cualquiera de ellas indica un potencial de riesgo alto:

- **Experiencia:** ¿Se ha trabajado con el cliente anteriormente?
- **Definición de Requisitos:** ¿Tiene el cliente una idea formal de lo que se requiere? ¿La ha **escrito** (documentado)?
- **Compromiso y Colaboración:** ¿Aceptará el cliente gastar su tiempo en **reuniones formales** de requisitos y revisiones? ¿Está dispuesto a establecer una comunicación fluida?
- **Interferencia Técnica:** ¿Dejará el cliente al personal técnico hacer su trabajo? (Es decir, ¿resistirá la tentación de **interferir** durante el trabajo técnico?).
- **Conocimiento del Proceso:** ¿Entiende el cliente el proceso del software que se está empleando?

- **Sustancia:** ¿Es técnicamente sofisticada el área del producto?

En resumen: Un **"mal cliente"** (caracterizado por falta de definición, compromiso o comprensión del proceso) representa un riesgo sustancial que debe ser investigado a fondo.

1.1. Riesgos del Proceso (6.3.4): Se centran en la definición y formalidad del proceso de software. El riesgo es alto si el proceso se realiza **"sobre la marcha"** y la calidad es solo una aspiración. Las preguntas clave se dividen en:

- **Aspectos del Proceso:** ¿Hay normas escritas y proceso estándar? ¿Se usa el mismo proceso en otros proyectos? ¿Hay formación y documentación sobre estándares de ingeniería del software? ¿Se realizan revisiones técnicas formales (requisitos, diseño, código)? ¿Existe gestión de configuración y control de cambios?
- **Aspectos Técnicos:** ¿Se emplean técnicas específicas de análisis, diseño y pruebas? ¿Se utilizan herramientas de software para apoyar la planificación, gestión de configuración, análisis, diseño y pruebas? ¿Se han establecido métricas de calidad y productividad?

1.2. Riesgos Tecnológicos (6.3.5): Se asocian con alcanzar los límites de la tecnología (**"tecnología de punta"**), lo cual es muy arriesgado y hace difícil la predicción de riesgos. El riesgo aumenta si:

- La tecnología o los **algoritmos** son nuevos para la organización.
- El software interactúa con **hardware nuevo o no probado** o con **productos de software de terceros no probados**.
- Los requisitos demandan **interfaces de usuario especiales** o componentes de programación no desarrollados previamente.
- Se requiere el uso de **métodos de desarrollo no convencionales** (ej. métodos formales, IA, redes neuronales).
- El cliente no está seguro de la **factibilidad** de la funcionalidad pedida o hay **restricciones de rendimiento** excesivas.

1.3. Riesgos del Entorno de Desarrollo (6.3.6): Un entorno de ingeniería de software inadecuado o ineficaz es una fuente de riesgo significativa. El riesgo es alto si:

- No se dispone de herramientas clave (gestión de proyectos, gestión del proceso, análisis, diseño, pruebas, gestión de configuración).
- Las herramientas de análisis y diseño no son apropiadas para el producto a construir.
- Las herramientas no están integradas entre sí.
- El personal no ha sido formado en las herramientas disponibles o no hay expertos para responder preguntas.

- La ayuda en línea y la documentación de las herramientas es inadecuada.

1.4. Riesgos del Personal (6.3.7): Sugiere preguntas para valorar el riesgo asociado a la plantilla de personal y su experiencia:

- ¿Se tiene al mejor personal? ¿Tienen los conocimientos adecuados?
- ¿Se tiene suficiente personal? ¿Ha sido asignado para toda la duración del proyecto?
- ¿Será mínimo el movimiento de personal (para asegurar la continuidad)?
- ¿Ha recibido el personal la formación y las expectativas correctas sobre el trabajo?

2. Componentes y Proyección del Riesgo

2.1. Componentes del Riesgo (Fuerzas Aéreas de EE. UU. - 6.3.8)

Se definen cuatro componentes de riesgo clave, cada uno midiendo un grado de incertidumbre:

- **Riesgo de Rendimiento:** Incertidumbre de que el producto cumpla sus requisitos y se adecúe al uso pretendido.
- **Riesgo de Coste:** Incertidumbre de que se mantendrá el presupuesto del proyecto.
- **Riesgo de Soporte:** Incertidumbre de la facilidad del software para ser corregido, adaptado y mejorado.
- **Riesgo de Planificación Temporal:** Incertidumbre de que se mantendrá la planificación temporal y se entregará a tiempo.

2.2. Proyección del Riesgo (Estimación) (6.4)

La proyección del riesgo busca cuantificar cada riesgo en dos dimensiones:

1. **Probabilidad:** La posibilidad de que el riesgo se haga real.
2. **Consecuencias:** La severidad del problema si el riesgo ocurre.

Cuatro Actividades: Establecer una escala de probabilidad, definir las consecuencias, estimar el impacto general y apuntar la exactitud de la proyección.

2.3. Desarrollo de una Tabla de Riesgo (6.4.1)

Es una técnica sencilla para la proyección y priorización del riesgo:

1. **Listar y Categorizar:** Se listan todos los riesgos (usando el checklist) y se categorizan (ej., PS para tamaño del proyecto, BU para negocio).

2. **Estimar Probabilidad:** Se obtiene un valor de probabilidad consensuado por el equipo (ej., haciendo una media de las estimaciones individuales).
3. **Valorar Impacto:** Se valora el impacto en cada uno de los cuatro componentes de riesgo (Rendimiento, Coste, Soporte, Planificación Temporal) usando una escala (ej., despreciable, marginal, crítico, catastrófico) y se promedia para obtener un valor general.
4. **Priorización:** La tabla se ordena por probabilidad y por impacto, colocando a los riesgos de alta probabilidad y alto impacto en la cima, lo que permite al jefe de proyecto definir una línea de corte para centrar los esfuerzos de mitigación.

Tipos de riesgos:

- Riesgos del proyecto
- Riesgos técnicos
- Riesgos del negocio
- Impredecibles
- Predecibles: hay que sentarse a pensar
- Conocidos

Actividades

1. Identificación/Análisis: **Lista de comprobación de los elementos:**
    a. Tamaño del producto
    b. Impacto empresarial
    c. Características de los participantes
    d. Definición del proceso
    e. Entorno de desarrollo
    f. Tecnología por construir
    g. Tamaño y experiencia del personal

    Componentes de riesgo:
    Rendimiento
    Costo
    Soporte
    Calendario
2. Proyección o estimación de riesgos: **Calificar el riesgo:**
    a. Probabilidad
    b. Impacto

    Pasos:
    1. Establecer una escala que refleje la probabilidad percibida
    2. Determinar las consecuencias
    3. Estimar el impacto sobre el proyecto y el producto

    ### Tabla de riesgos

Riesgos	Categoría	Probabilidad	Impacto	RMMM
Estimación de tamaño puede ser significativamente baja	PS	60%	2	
Mayor número de usuarios que el planificado	PS	30%	3	
Menos reuso que el planificado	PS	70%	2	
Usuarios finales que se resisten al sistema	BU	40%	3	
Fecha de entrega será apretada	BU	50%	2	
Pérdida de fondos	CU	40%	1	
Cliente cambiará requisitos	PS	80%	2	
Tecnología no satisfará las expectativas	TE	30%	1	
Falta de capacitación en herramientas	DE	80%	3	
Personal inexperto	ST	30%	2	
Alta rotación de personal	ST	60%	2	
Σ				
Σ				
Σ				

3. Supervisión: **Estrategia para:**
    a. Mitigar
    b. Monitorear
    c. Gestionar el riesgo y planificar la contingencia

    Pasos:
    1. Establecer una escala que refleje la probabilidad percibida
    2. Determinar las consecuencias
    3. Estimar el impacto sobre el proyecto y el producto
4. Evaluación: **Estrategia para:**
    a. Mitigar
    b. Monitorear
    c. Gestionar el riesgo y planificar la contingencia
5. **Supervisión:**

Pasos:

1. Establecer una escala que refleje la probabilidad percibida
2. Determinar las consecuencias ,
3. Estimar el impacto sobre el proyecto y el producto

Resumen del Análisis y Gestión del Riesgo en Software (Parte III: Proyección y Estrategia)

6.4.2. Proyección y Evaluación del Riesgo (Estimación): La proyección (o estimación) del riesgo busca medir la probabilidad y las consecuencias de un riesgo si se materializa.

- Factores que Afectan las Consecuencias:
    - **Naturaleza:** Los problemas probables que surgirían (ej., una interfaz mal definida impide la integración).
    - **Alcance:** La combinación de la severidad (qué tan serio) y su distribución (a qué parte del proyecto o cuántos clientes afectará).

    - **Temporización:** Cuándo ocurrirá y por cuánto tiempo se sentirá el impacto (generalmente, las malas noticias se prefieren cuanto antes).
- Proceso Iterativo: La proyección y el análisis se aplican **reiteradamente**. El equipo debe **revaluar** la tabla de riesgo periódicamente, añadiendo, eliminando o cambiando la prioridad de los riesgos.
- 6.4.3 Evaluación y Nivel de Referencia: Durante la evaluación se define un nivel de referencia de riesgo. Este es el punto (o punto de ruptura) en el que la combinación de problemas (coste, rendimiento, etc.) es tan grave que la decisión de seguir o abandonar el proyecto son igualmente aceptables.

6.5 Reducción, Supervisión y Gestión del Riesgo (RSGR)

Una estrategia proactiva de riesgo eficaz tiene tres componentes principales:

Componente	Objetivo Principal	Acciones Clave
Reducción (Evitar)	Prevenir que el riesgo se convierta en problema, desarrollando un plan de evitación.	Identificar y eliminar las causas de riesgo (ej., si la movilidad del personal es alta, mejorar las condiciones laborales) y desarrollar técnicas de continuidad (ej., documentar todo el trabajo, definir reservas).
Supervisión	Seguir los factores que pueden indicar si un riesgo se está volviendo más o menos probable.	Monitorear factores indicadores (ej., actitud del equipo, relaciones interpersonales) y la efectividad de los pasos de reducción que ya se implementaron. También se utiliza para determinar el "origen" (qué riesgos causaron un problema).
Gestión y Contingencia	Preparar un plan de respuesta asumiendo que el riesgo ha fallado.	Asignar temporalmente reservas, dispersar el conocimiento, reasignar recursos, y pedir a las personas que se van que transfieran sus conocimientos (vídeos, reuniones).

- Regla de Pareto (80/20): Para evitar que la gestión del riesgo se convierta en un proyecto en sí mismo, se aplica la regla 80/20: el 80% del riesgo total se debe solamente al 20% de los riesgos identificados. El Plan RSGR debe enfocarse en ese 20% crítico de riesgos de alta prioridad.
- Análisis Costo/Beneficio: Los pasos de RSGR aumentan el coste y la duración del proyecto. La gestión debe evaluar si los beneficios de evitar el riesgo superan los costes de la implementación de la estrategia.

6.6.. Seguridad y Peligros

- **Riesgos Post-Entrega:** El riesgo no termina con el proyecto; puede ocurrir **después de la entrega** si un fallo en el software provoca pérdidas económicas o daños físicos (ej., en sistemas de seguridad crítica).
- **Análisis del Peligro:** El análisis de seguridad del software y del peligro es una actividad de calidad que busca identificar peligros potenciales al inicio del proceso. Esto permite especificar características de diseño que eliminen o controlen esos peligros.

6.7 El Plan RSGR (Estructura Formal): La estrategia de gestión de riesgo se puede formalizar en un Plan de Reducción, Supervisión y Gestión del Riesgo (Plan RSGR), que forma parte del Plan del Proyecto general.

Estructura Típica del Plan RSGR:

1. **Introducción:** Alcance, propósito, visión general de riesgos principales y responsabilidades.
2. **Tabla de Riesgo del Proyecto:** Riesgos por encima de la línea de corte y factores que influyen.
3. **Estrategia RSGR Detallada (por riesgo):**
    - **Reducción:** Estrategia y pasos específicos.
    - **Supervisión:** Factores y enfoque de monitoreo.
    - **Gestión:** Plan de contingencia.
4. **Planificación Temporal:** Frecuencia de revisión del Plan RSGR.

6.8. Resumen: El análisis de riesgos es una inversión de tiempo que se justifica al reducir los trastornos, aumentar la capacidad de control del proyecto y proporcionar la confianza de planificar los problemas antes de que ocurran. Como dijo Sun Tzu, "Si conoces al enemigo [el riesgo] y te conoces a ti mismo, no tendrás que temer el resultado de cien batallas".

Calidad

Todo lo que hacemos en éste ámbito es en pos de buscar construir aplicaciones de calidad

¿Qué es?

- Satisfacción del usuario = producto que funciona + buena calidad + entrega dentro del presupuesto y plazo.
- Conformidad con los requisitos y confianza en el funcionamiento.
- La suma de todos aquellos aspectos o características de un producto o servicio que influyen en su capacidad para satisfacer las necesidades,expresadas o implícitas.
- Adecuación para su uso.
- Grado con el cual el cliente o usuario percibe que el software satisface sus expectativas.
- Capacidad del producto software para satisfacer los requisitos establecidos.

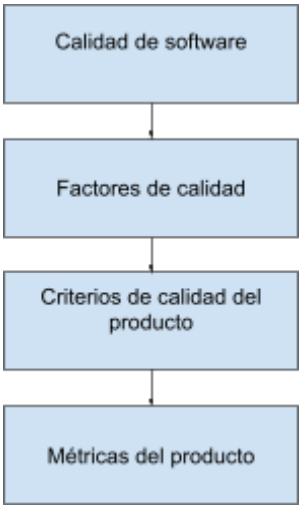
Calidad de Software: “Proceso eficaz de software que se aplica de manera que crea un producto útil que proporciona valor medible a quienes lo producen y a quienes lo utilizan.”

Existen 2 tipos de calidad:

Calidad de producto: La **Calidad del Producto** se concentra en el **resultado final** y las **propiedades intrínsecas** del *software* tal como lo ve el usuario, midiendo si el sistema cumple con todos sus requisitos funcionales (corrección, fiabilidad, eficiencia) y satisface las expectativas del cliente.

Calidad de proceso: La Calidad del Proceso se concentra en el método de trabajo o la receta utilizada para crear el *software*, buscando que las actividades, las prácticas y las metodologías de desarrollo sean consistentes, controladas y repetibles. Este enfoque es preventivo, ya que un proceso de alta calidad es el factor más importante para garantizar que el producto final tenga inherentemente menos defectos y cumpla con los estándares.

Modelos de calidad



El modelo de MeCall

El Modelo de Calidad de McCall (1977) es uno de los primeros modelos de calidad de *software* orientados a atributos. Su principal objetivo es unir la perspectiva del usuario (requisitos de alto nivel) con la perspectiva del desarrollador (atributos técnicos de bajo nivel).

Los Tres Puntos de Vista(Ejes de Calidad): Estos tres ejes representan las **tres grandes preguntas** que el usuario y el gestor se hacen sobre el producto *software* en diferentes momentos de su ciclo de vida.

Punto de vista	Factores
Operación del producto: Se enfoca en la funcionalidad inmediata y las características del producto cuando está siendo utilizado o ejecutado.	Facilidad de uso Integridad Corrección Fiabilidad Eficiencia
Revisión del producto: Se enfoca en la capacidad de modificación y prueba del *software* a lo largo de su vida útil. Es vital para el mantenimiento.	Facilidad de mantenimiento Facilidad de prueba Flexibilidad
Transición del producto: Se enfoca en la adaptabilidad del *software* para ser transferido o interactuar con otros sistemas o plataformas.	Facilidad de reutilización Interoperabilidad Portabilidad

Factores de la calidad

Los 11 Factores de Calidad son los atributos de alto nivel del producto. Estos son los aspectos que el usuario o el gestor pueden observar y que determinan el éxito del producto.

Conceptualmente: Los factores son las metas de calidad que se buscan. Son conceptos abstractos (ej., "Fiabilidad", "Portabilidad") que se agrupan bajo los tres ejes.

Los Criterios de Calidad son los atributos de bajo nivel y son de naturaleza técnica.

- **Conceptualmente:** Los criterios son las medidas o las propiedades internas del código y la arquitectura. Son los elementos que el desarrollador controla para influir en los factores.
- **Relación Factor-Criterio:** Un solo criterio puede afectar a varios factores. Por ejemplo, la Modularidad (un criterio) mejora la Facilidad de mantenimiento, la Reutilización y la Facilidad de prueba (varios factores).

Factor - Criterios de calidad de McCall

Factor	Criterio
Facilidad de uso	• Facilidad de operación • Facilidad de comunicación. • Facilidad de aprendizaje.
Usabilidad	• Facilidad de operación • Facilidad de aprendizaje
Integridad	• Control de accesos • Facilidad de auditoría |
Corrección	• Completitud • Consistencia • Trazabilidad
Fiabilidad	• Precisión • Consistencia • Tolerancia a fallos • Modularidad • Simplicidad
Confiabilidad	• Consistencia • Tolerancia a fallos

Eficiencia	• Eficiencia de ejecución • Eficiencia en almacenamientos
Facilidad de mantenimiento	• Modularidad • Simplicidad • Consistencia • Decisión • Autodescripción
Facilidad de prueba	• Modularidad • Simplicidad • Autodescripción • Instrumentación
Flexibilidad	• Autodescripción • Capacidad de expansión • Generalidad • Modularidad
Reusabilidad	• Autodescripción • Generalidad • Modularidad • Independencia del hardware • Independencia Sistema / Software
Interoperabilidad	• Modularidad • Compatibilidad de comunicaciones • Compatibilidad de datos
Portabilidad	• Autodescripción • Modularidad • Independencia Sistema / Software • Independencia del hardware

El dilema de la calidad

Construcción artesanal:

- Basado sólo en las capacidades individuales.
- Sin procesos de construcción de software.

Burocracia extrema:

- Procesos demasiado rígidos.
- Infinidad de roles.
- Demasiadas actividades de validación

¿Cómo se logra la calidad?

1. Métodos de la ingeniería del software.
2. Administrar bien el proyecto.
3. Realizar un control de calidad exhaustivo.
4. Contar con infraestructura de aseguramiento de calidad.
5. Control de la calidad.

Control de la calidad

El objetivo de las actividades de control de calidad es comprobar si un producto posee o no una determinada característica de calidad en el grado requerido.

Actividades de control de calidad:

a. Controles Estáticos (Revisiones)

b. Controles Dinámicos (Prueba del software)

Tipos de pruebas:

- Prueba modular
- Prueba de integración
- Prueba de sistema
- Prueba de aceptación
- Prueba de regresión

Aseguramiento de la calidad

Actividades específicas del proceso que permiten a una organización de software garantizar que hace “las cosas correctas en el momento correcto y de la forma correcta”

¿Cuando se hace?¿Quien la hace?

Elementos:

- Estándares
- Revisiones y auditorías
- Pruebas
- Colección y análisis de errores
- Administración del cambio
- Educación
- Administración de proveedores
- Administración de la seguridad
- Administración del riesgo

Sistema de calidad: Es el que define cómo implementar la garantía o aseguramiento de la calidad.

Métricas

Medición:

- Acción o acto
- Proceso mediante el cual se asignan número o símbolos a los atributos de las entidades del mundo real.

Medida:

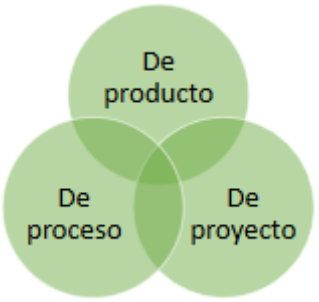
- Proporciona un indicio cuantitativo de la extensión, cantidad, dimensión, capacidad o tamaño de algún atributo de un producto o proceso.

Métrica:

- Medida cuantitativa del grado en el que un sistema, componente o proceso posee un atributo determinado
- Relaciona de alguna forma las medidas individuales.

Indicador:

- Conjunto de métricas que proporcionan comprensión acerca del proceso, proyecto o producto software.
- Permite a los gerentes de proyectos aplicar ajustes para hacer las cosas mejor.



PROCESO DE SOFTWARE Y MÉTRICAS DE PROYECTOS (anexo)

La medición es una parte fundamental de cualquier disciplina de ingeniería, incluida la del software. Como menciona Lord Kelvin, **medir y expresar con números** nos da un conocimiento objetivo y sólido, mientras que sin medición, el conocimiento es deficiente.

En el contexto del software, la medición se conoce como **métricas del software**. Se usan para obtener una visión profunda de la eficacia de los procesos y proyectos. Esto ayuda a los equipos a mejorar la calidad y la productividad.(fin)

4.1 MEDIDAS ,metricas e INDICADORES(anexo)

Aunque a menudo se usan indistintamente, estos términos tienen diferencias sutiles pero importantes en la ingeniería de software:

- **Medida:** Es una **indicación cuantitativa simple**. Por ejemplo, el número de errores encontrados en un módulo. Es el dato crudo.
- **Medición:** Es el **acto de determinar una medida**. Se refiere al proceso de recopilar los datos. Por ejemplo, investigar varias revisiones para contar los errores.
- **Métrica:** Es una **medida cuantitativa** que relaciona datos individuales para proporcionar un contexto. Por ejemplo, el número de errores encontrados por

persona-hora de revisión. Esta relación es lo que la hace útil.

- **Indicador:** Es una métrica o una combinación de métricas que ofrece una **visión profunda** del proyecto, proceso o producto. Permite a los gestores tomar **decisiones fundamentadas** para mejorar. El texto da un excelente ejemplo: una métrica de "errores por persona-hora" se convierte en un indicador cuando revela que los equipos que usan revisiones formales son más eficientes. Esto sugiere una mejora para todo el equipo.(fin)

Vistazo Rápido a las Métricas)(ANEXO)

El texto ofrece un resumen muy útil en la sección de "vistazo rápido" que podemos sintetizar así:

- **¿Qué son?** Las métricas son medidas cuantitativas de la calidad y productividad del software.
- **¿Quién las usa?** Los **administradores de software** las analizan y evalúan, mientras que los **ingenieros de software** las recopilan.
- **¿Por qué son importantes?** Permiten dejar de lado los juicios subjetivos para poder identificar **tendencias**, hacer mejores **estimaciones** y lograr una **mejora real** con el tiempo.
- **¿Cómo se aplican?** Se definen un conjunto de medidas fáciles de recopilar, se normalizan (ej. por tamaño o función), se analizan y se comparan con promedios de proyectos anteriores para sacar conclusiones.
- **¿Qué se obtiene?** Un conjunto de métricas que ofrecen una **visión profunda** de los procesos y proyectos.
- **¿Cómo se usan correctamente?** Aplicando un plan de medición consistente, sin usar los datos para castigar o premiar el rendimiento individual.

Razones para Medir: El Enfoque de la Guía

El documento también destaca cuatro razones clave por las que medimos, según Park, Goethert y Florac:

- **Caracterizar:** Para entender mejor los procesos, productos y recursos, y establecer **líneas base** para futuras comparaciones.
- **Evaluar:** Para determinar el estado de un proyecto o proceso y saber si nos estamos desviando del plan.
- **Predecir:** Para poder **planificar** de manera más precisa y gestionar el **rr Riesgo**: Usamos datos históricos para estimar costos, tiempos y calidad.
- **Mejorar:** Para identificar y corregir ineficiencias, cuellos de botella y problemas que afectan la calidad del producto y el rendimiento del proceso.

En resumen, la medición en la ingeniería de software es la herramienta que permite a los equipos y gestores tomar decisiones basadas en datos, en lugar de en suposiciones o intuiciones.

Métricas de producto

1. Métricas para el modelo de requerimientos
    a. Métricas basadas en funciones (puntos de función)
        b. Métricas para la calidad de la especificación

2. Métricas para el modelo de diseño
    a. Métricas orientadas a clase

4.5 Métricas para la Calidad del Software: (anexo)

El objetivo de la ingeniería de software es la alta calidad, y las métricas son esenciales para medirla de forma objetiva. El primer paso es medir los errores y defectos.

Los factores de calidad (propuestos por McCall y Cavano en 1978) siguen siendo relevantes. Estos evalúan el software desde tres perspectivas:

- **Operación del producto:** Calidad mientras se usa (ej: corrección y fiabilidad).
- **Revisión del producto:** Facilidad para cambiarlo (ej: mantenibilidad y flexibilidad).
- **Transición del producto:** Facilidad para adaptarlo a otro entorno (ej: portabilidad).

El texto propone métricas para medir cuatro atributos clave de la calidad:

- **Corrección:** Se mide por el número de defectos por cada mil líneas de código (KLDC).
- **Mantenibilidad:** Se mide indirectamente. Una métrica común es el Tiempo Medio de Cambio (TMC), que es el tiempo necesario para analizar, implementar, probar y distribuir un cambio.
- **Integridad:** Mide la capacidad del sistema para resistir ataques. Se calcula a partir de la probabilidad de amenaza y la probabilidad de seguridad.
- **Facilidad de uso:** Se mide por el tiempo que se tarda en aprender a usar el sistema, el aumento de productividad que genera y la satisfacción subjetiva de los usuarios.

Un indicador de calidad muy importante es la Eficacia de la Eliminación de Defectos (EED), que se calcula como:

EED = E / (E + D)

Donde E son los errores encontrados antes de la entrega, y D son los defectos encontrados después. Un valor cercano a 1 indica que el equipo es muy eficaz en encontrar y corregir errores antes de que el software llegue al cliente.(fin)

Métricas Orientadas al Tamaño (LDC):

Estas métricas normalizan las medidas de calidad y productividad en función del "tamaño" del software, medido en **Líneas de Código (LDC)**.

Para calcularlas, se recopilan datos del proyecto como:

- LDC producidas.
- Esfuerzo (personas-mes) y costo.
- Errores y defectos.
- Páginas de documentación.

Luego se derivan métricas como:

- Errores por miles de LDC (KLDC).
- LDC por persona-mes.
- Costo por LDC.(FIN)

Métricas de proceso y proyecto

4.2Métricas en el Proceso y en el Proyecto 🚀 (anexo)

Las métricas de software se aplican en dos dominios principales:

- **Dominio del Proceso:** Las métricas del proceso se recopilan a lo largo de muchos proyectos y durante un período prolongado. Su objetivo es evaluar la **eficacia del proceso** de software de la organización y proporcionar indicadores para **mejoras a largo plazo**. Es el único factor controlable para mejorar la calidad y el rendimiento de una organización, aunque la **habilidad y motivación del personal** son los factores más influyentes.
- **Dominio del Proyecto:** Las métricas de proyecto se utilizan para que el gestor pueda **evaluar el estado del proyecto en curso**, detectar riesgos, identificar problemas y evaluar la calidad del trabajo del equipo.

Es importante destacar que las mismas medidas pueden usarse para ambos

dominios. Los datos de un proyecto individual pueden convertirse en métricas de proceso cuando se analizan en conjunto con datos de otros proyectos.

Métricas de Proceso y Mejoras Continuas(parte) inicio:

La eficacia de un proceso de software se mide de forma **indirecta**, evaluando los resultados que genera. Estos resultados incluyen datos sobre errores, defectos, productividad, esfuerzo y tiempo.

Las métricas del proceso se pueden clasificar según su nivel de uso:

- **Métricas privadas:** Estos datos, como la cantidad de defectos por módulo, son para uso individual. Sirven para que cada ingeniero pueda evaluar y mejorar su propio rendimiento, de acuerdo con el **Proceso Personal de Software (PPS)** de Humphrey. La idea es que cada persona encuentre los métodos que le funcionen mejor.
- **Métricas de equipo:** Son privadas para el equipo del proyecto. Permiten al equipo revisar su propio rendimiento, identificar áreas de mejora y optimizar su trabajo en conjunto.
- **Métricas públicas:** Recopilan información de proyectos y equipos (sin atribuirla a un individuo) para que la organización pueda tomar **decisiones estratégicas** que mejoren el proceso de software a nivel general.

Métricas de Proyecto: Las métricas de proyecto tienen un propósito **táctico**, es decir, son utilizadas por el gestor y el equipo del proyecto para adaptar el flujo de trabajo y las actividades técnicas. Su principal objetivo es:

1. **Minimizar los retrasos y riesgos** ajustando la planificación y el esfuerzo.
2. **Evaluar la calidad** de los productos de trabajo a medida que el proyecto avanza. A medida que la calidad mejora, los defectos disminuyen, lo que reduce el trabajo de rehacer y, por ende, el costo total del proyecto.

Las métricas de proyecto miden las **entradas** (recursos), las **salidas** (productos entregables) y los **resultados** (la efectividad de las entregas).

4.3 Mejoras en el Proceso y Uso Ético de las Métricas

La **mejora estadística del proceso del software (MEPS)** es un enfoque riguroso que utiliza el análisis de errores y defectos. Se categorizan, se registran sus costos y se identifican las categorías más costosas para luego modificar el proceso y reducir su frecuencia. Esto ayuda a comprender las fortalezas y debilidades del proceso.

El texto también subraya la importancia de la **ética en el uso de las métricas**:

- No usar métricas para evaluar o castigar a individuos.
- Proporcionar retroalimentación regular y trabajar en equipo para establecer objetivos.
- No obsesionarse con una sola métrica.
- Ver los datos que señalan problemas como indicadores para la mejora, no como algo "negativo".

En esencia, la medición debe ser una herramienta para el crecimiento y no para la

vigilancia o el castigo.

Guías para Usar Métricas de Forma Correcta:

Las pautas clave son:

- Usar el **sentido común** al interpretar los datos.
- Dar **retroalimentación** constante a los equipos.
- **No usar las métricas para evaluar o castigar** el rendimiento individual.
- Trabajar con los equipos para establecer **objetivos claros**.
- Ver los datos que señalan problemas como **oportunidades de mejora**, no como algo negativo.
- **No obsesionarse con una sola métrica**, sino ver el panorama completo.

Mejora Estadística del Proceso de Software (MEPS)

El **MEPS** es un enfoque más riguroso que utiliza el análisis de fallos para mejorar el proceso. Se basa en una metodología de seis pasos:

1. **Categorizar** todos los errores y defectos por su origen (ej: errores de lógica, de especificación).
2. **Registrar** el costo de corregir cada uno.
3. **Contar** y ordenar las categorías de errores por frecuencia.
4. **Computar** el costo total de cada categoría.
5. **Analizar** los datos para identificar las clases de errores más costosas.
6. **Desarrollar planes** para modificar el proceso y reducir la aparición de esos errores.

El **diagrama de espina** es una herramienta visual utilizada en este proceso para diagnosticar la **causa raíz** de un problema. La "espina" representa el problema principal (ej: defectos de especificación), y las "varillas" representan las posibles causas (ej: requisitos ambiguos, cambios del cliente).

La medición y el análisis de datos son cruciales para entender y mejorar el proceso de software, permitiendo a la organización fortalecer sus puntos débiles de manera sistemática.

4.6 Integración de Métricas en el Proceso 📈

A pesar de la resistencia inicial, la integración de métricas en el proceso de desarrollo de software es crucial para la mejora continua.

Argumentos para medir:

- Para la gerencia: La medición permite establecer una línea base de productividad y calidad para luego fijar objetivos realistas de mejora.
- Para los proyectos: Permite hacer mejores estimaciones y monitorear el progreso del proyecto en tiempo real.
- Para los técnicos: Las métricas ayudan a identificar qué módulos son más propensos a errores y a planificar las pruebas de manera más efectiva.

Para establecer una línea base de métricas, los datos recopilados de proyectos anteriores deben ser exactos, de una muestra grande, consistentes y de proyectos similares.

El proceso de las métricas sigue tres pasos:

1. Colección: Recopilación de medidas.
2. Cómputo: Cálculo de las métricas.
3. Evaluación: Análisis de las métricas para obtener indicadores que guíen las decisiones.

Un método destacado para la integración de métricas es el Objetivo, Pregunta y Métrica (OPM) de Victor Basili. Este enfoque se basa en definir claramente los objetivos (ej: reducir el trabajo de revisión), formular preguntas para aclarar esos objetivos (ej: ¿cómo cuantificar el trabajo de revisión?) y luego desarrollar las métricas necesarias para responderlas. Este método utiliza plantillas para definir el propósito, la perspectiva (ej: del cliente o del diseñador) y el entorno del análisis.(FIN)

Medidas directas e indirectas

Medidas directas del proceso: Costo y esfuerzo aplicado

Medidas directas del producto: LOC construidas

Medidas indirectas del producto: Funcionalidad, calidad, complejidad, eficiencia, confiabilidad, etc.

Tipos de Mediciones de Software:

Las mediciones del software se clasifican de manera similar a las del mundo físico:

- **Medidas directas:** Son fáciles de obtener. Incluyen el **costo**, el **esfuerzo**, las **líneas de código (LDC)**, la velocidad de ejecución y los defectos informados.
- **Medidas indirectas:** Son más difíciles de evaluar, ya que se basan en la percepción de atributos. Incluyen la **calidad**, la **funcionalidad**, la **complejidad**, la eficiencia y la fiabilidad.

Las métricas, a su vez, se normalizan para permitir comparaciones entre diferentes proyectos. Esta normalización se puede hacer de dos maneras.

Basadas en funciones

			Factor ponderado				
Valor de dominio de información	**Conteo**		**Simple**	**Promedio**	**Complejo**		
Entradas externas (EE)		X	3	4	6	=	
Salidas externas (SE)		X	4	5	7	=	
Consultas externas (CE)		X	3	4	6	=	
Archivos lógicos internos (ALI)		X	7	10	15	=	
Archivos de interfaz externos (AIE)		X	5	7	10	=	
						Conteo total	

$$PF = \text{Conteo total} \times [0{,}65 + 0{,}01 \times \sum(Fi)]$$

Archivo XLSX

Métricas Orientadas a la Función (PF)(anexo): Estas métricas normalizan las medidas según la **funcionalidad** entregada por la aplicación, usando una medida

llamada **Puntos de Función (PF)**.

Para calcular los PF se siguen dos pasos:

1. **Conteo:** Se cuentan cinco características del dominio de información del software (entradas, salidas, peticiones, archivos e interfaces externas) y se les asigna un peso de complejidad (simple, medio, complejo).
2. **Ajuste:** Se aplica un **factor de ajuste de complejidad** basado en 14 preguntas sobre las características del sistema (ej: fiabilidad, rendimiento, comunicaciones). Este proceso puede ser subjetivo.

Una vez que se tienen los PF, se pueden calcular métricas como:

- Errores por PF.
- PF por persona-mes.
- Costo por PF.

Extensiones de Puntos de Función: El método de PF original era limitado para sistemas de ingeniería o de tiempo real, que tienen una alta complejidad algorítmica y de control. Para solucionar esto, se crearon extensiones:

- **Puntos de Características:** Extienden los PF para incluir la **complejidad de los algoritmos**.
- **Puntos de Función 3D:** Integran las dimensiones de **datos, función y control**, permitiendo una medición más completa de la funcionalidad en sistemas complejos.

Tanto las LDC como los PF son métricas controvertidas. Los defensores de los PF afirman que son independientes del lenguaje de programación, mientras que los detractores señalan su subjetividad y la dificultad para recopilar los datos.(fin)

Calidad de la especificación

Lista de características para valorar la calidad del modelo de requerimientos (cualitativas):

- Especificidad (falta de ambigüedad)
- Completitud
- Corrección
- Comprensibilidad
- Verificabilidad
- Consistencia interna y externa
- Factibilidad
- Concisión
- Rastreabilidad
- Modificabilidad

- Precisión
- Reusabilidad

Cada una de las características puede representarse usando una o más métricas. Por ejemplo:

La fórmula es:

EED=E/(E+D)

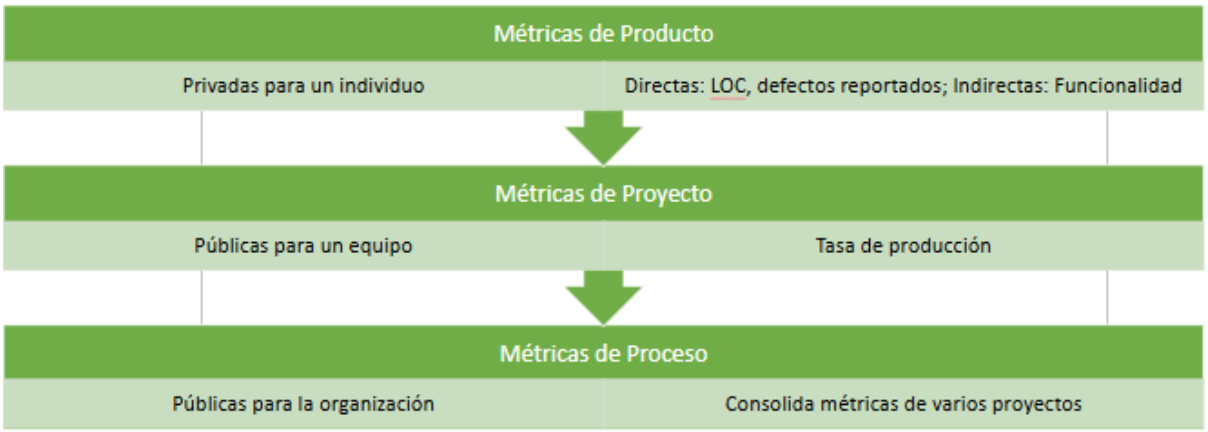
Donde:

- **E** es el número de errores encontrados antes de entregar el software a su usuario final.
- **D** es el número de defectos encontrados después de la entrega.

Métricas orientadas a clase

Métodos ponderados por clase (MPC):Profundidad de árbol de herencia (PAH): La máxima longitud desde el nodo hasta la raíz del árbol. (fin parte 2)

Producto, Proyecto y Proceso



Reconciliación de los Enfoques: LDC vs. Puntos de Función 🤝(ANEXO)

La relación entre las Líneas de Código (LDC) y los Puntos de Función (PF) depende del lenguaje de programación. Por ejemplo, una LDC en un lenguaje moderno como Visual Basic puede proporcionar más funcionalidad que una en FORTRAN o C.

Lenguaje de Programación	LDC/PF (media)
Ensamblador	320
C++	64
Visual Basic	32
SQL	12

Es crucial no usar estas métricas para comparar la productividad de individuos o equipos, ya que muchos factores pueden influir en el resultado. Sin embargo, si se usan con una **línea base histórica**, tanto las LDC como los PF pueden ser predictores precisos del esfuerzo y el costo del proyecto.

Metricas orientadas al tamaño

Proyecto	LOC	Esfuerzo	U$S (miles)	Páginas Doc.	Errores	Defectos	Personal
Proyecto X	12.100	24	168	365	134	29	3
Proyecto Y	27.200	62	440	1224	321	86	5
Proyecto Z	20.200	43	314	1050	256	64	6

1. Errores por KLOC
2. Defectos por KLOC
3. U$S por KLOC
4. Páginas de documentación por KLOC

Métricas orientadas en puntos de función

Usan una medida de la funcionalidad como valor de normalización

Ventajas: Independiente del lenguaje

Desventajas:Utiliza valores subjetivos

Combinación LOC y PF

	LOC por punto de función			
Lenguaje de programación	Promedio	Mediano	Bajo	Alto
C	162	109	33	704
C++	66	53	29	178
Java	63	53	77	-
Javascript	58	63	42	75
JSP	59	-	-	-
Smalltalk	26	19	10	55
SQL	40	37	7	110
Visual Basic	47	42	16	158

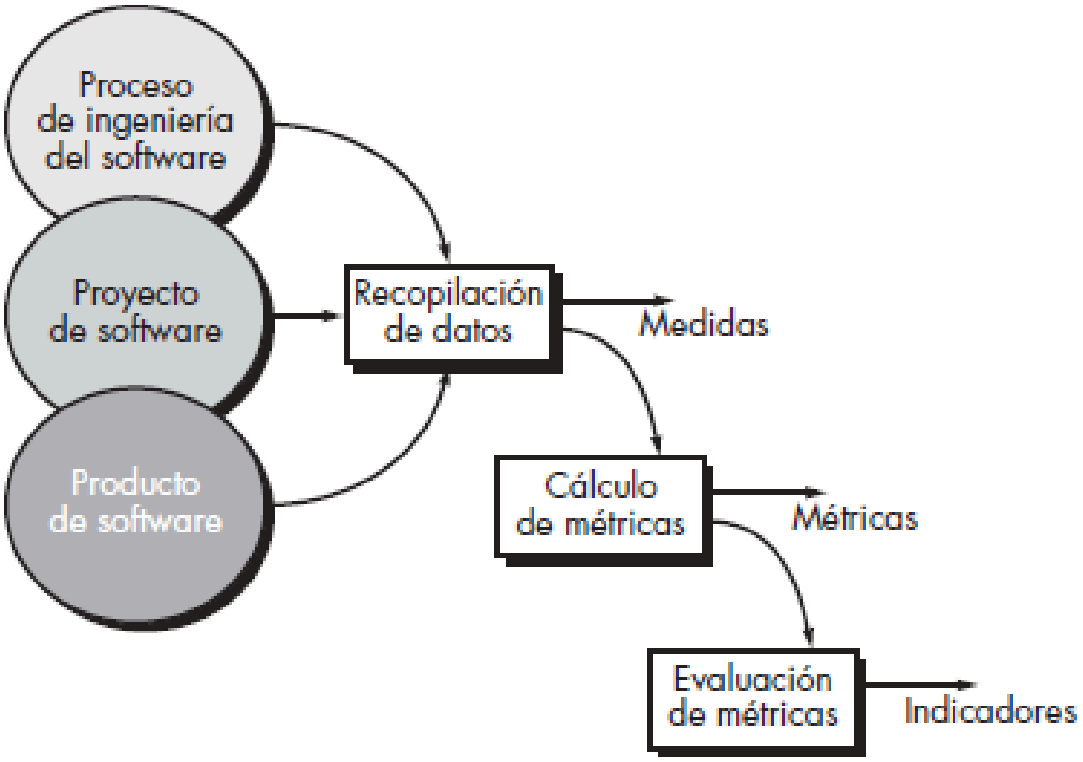
Métricas para la calidad del software

Medidas de la calidad de software	Descripción	Métrica
Exactitud	Grado en que un programa realiza la acción requerida	Defectos por KLOC (medidos en un período de tiempo)
Capacidad de mantenimiento	Facilidad con la que un programa puede corregirse	Tiempo medio al cambio
Integridad	Habilidad de un sistema para recibir ataques	Integridad = ∑[1 – (amenaza X (1 – seguridad))] *
Usabilidad	Facilidad para el uso de un sistema	Tiempo promedio de aprendizaje

Eficiencia de la remoción de defecto

$$ERD = \frac{E}{E + D}$$

Proceso de recopilación de métricas



Resumen

Definiciones	Tipos de métricas	Métricas de producto	Métricas de proyecto	Métricas de proceso
• Medición • Medidas • Métricas • Indicadores • Factor de normalización	• De proceso (estratégicas) • De proyecto (tácticas) • De producto	• Basadas en funciones • Para la calidad de la especificación • Orientadas a clase	• Orientadas al tamaño • Orientadas a función	• De calidad de software • Línea de referencia

Resumen de Métricas de Software

La **medición** en la ingeniería de software es fundamental porque permite a los equipos mejorar procesos, planificar, y controlar proyectos de manera objetiva. Al recolectar **medidas** (datos crudos) se pueden calcular **métricas**, y al analizar estas, se obtienen **indicadores** para guiar decisiones de gestión y técnicas.

Tipos y Usos de las Métricas

- **Métricas de proceso:** Tienen un objetivo **estratégico**. Permiten a una organización entender la efectividad de sus procesos a largo plazo y mejorarlos.
- **Métricas de proyecto:** Son de naturaleza **táctica**. Ayudan a los gestores a adaptar el trabajo del proyecto en tiempo real, facilitando el seguimiento y

control.

Las métricas también se clasifican por su enfoque:

- **Orientadas al tamaño:** Usan las **Líneas de Código (LDC)** como base para normalizar otras medidas, como el esfuerzo o la cantidad de defectos.
- **Orientadas a la función:** Utilizan los **Puntos de Función (PF)**, que miden la funcionalidad del software de manera indirecta, para normalizar las métricas.

Calidad y Validación: Las métricas de **calidad** son clave para identificar y corregir las causas de los defectos. Al crear una **línea base** de métricas de calidad, una organización puede evaluar su proceso y actuar para mejorarlo.

Para que las métricas sean válidas, deben someterse a un **análisis estadístico**. El **gráfico de control** es una herramienta útil que permite examinar la variación en los datos y determinar si las tendencias observadas son significativas o simplemente ruido estadístico.

Implementación: Implementar un programa de métricas es un cambio cultural que implica tres pasos principales:

1. **Recopilación de datos.**
2. **Cálculo de métricas.**
3. **Evaluación de las métricas** para obtener indicadores.

Un enfoque orientado a objetivos (como el OPM del texto anterior) ayuda a una organización a centrarse en las métricas más relevantes para su negocio. Al crear una línea base de métricas, que es una base de datos de mediciones pasadas, los equipos y gestores obtienen una visión profunda del trabajo que realizan y del producto que crean.

Establecimiento de un Programa de Métricas (4.10)

Esta sección describe una guía del Instituto de Ingeniería del Software (IIS) para crear un programa de métricas que esté **orientado a objetivos**. La idea principal es que las métricas que elijas deben estar directamente ligadas a las metas de negocio o técnicas que quieres lograr.

El proceso se basa en un refinamiento paso a paso:

1. Comienzas con un **objetivo de negocio** de alto nivel (ej. "mejorar la satisfacción del cliente").
2. Traduces ese objetivo en **preguntas** más específicas sobre las entidades del proyecto (recursos, código, etc.). Por ejemplo, "¿nuestros clientes reciben la información que necesitan en las solicitudes de cambio?".
3. Finalmente, desarrollas las **métricas** necesarias para responder a esas preguntas.

Este enfoque se conoce como **Objetivo, Pregunta y Métrica (OPM)**. El OPM utiliza plantillas para definir el **propósito**, la **perspectiva** (quién evalúa, ej. el gestor o el cliente) y el **entorno** (el contexto del proyecto). Esto asegura que cada métrica recolectada tenga una razón de ser y contribuya a un objetivo claro.(FIN)

Métricas para Organizaciones Pequeñas (4.9) (ANEXO)

Para las pequeñas empresas, la clave es que la implementación de métricas sea **simple** y **adaptada** a sus necesidades específicas. Un buen enfoque es comenzar con un objetivo de mejora sencillo, como "reducir el tiempo para implementar cambios". Con esto en mente, se recolectan unas pocas medidas fáciles de obtener (ej. tiempo y esfuerzo en las distintas fases del proceso de cambio).

A partir de estas medidas, se pueden calcular métricas simples, como el porcentaje de tiempo dedicado a cada fase. Esto ayuda a las empresas pequeñas a identificar dónde están los cuellos de botella y a mejorar su eficiencia de forma incremental. El costo de este programa de métricas es muy bajo y representa una inversión que se paga sola.

GESTIÓN DE CONFIGURACIÓN DEL SOFTWARE

¿Qué es la Gestión de Configuración del Software (GCS)?

La Gestión de Configuración del Software (GCS) es una disciplina de control fundamental cuya misión es manejar la evolución de un sistema de software. Su propósito, según Babich, es identificar, organizar y controlar todas las modificaciones que el software sufre durante el desarrollo y mantenimiento. Esto se realiza para maximizar la productividad de un equipo de programación mientras se minimizan los errores y la confusión. La GCS opera a lo largo de todo el ciclo de vida del producto, desde el inicio del desarrollo hasta que el software se retira.

¿Qué papel juega la Gestión de Configuración dentro de un Proyecto de Desarrollo de Software?

El éxito de un proyecto depende de la correcta ejecución de 4 tipos de funciones:

- **Gestión del Proyecto:** Se enfoca en las tareas administrativas y de liderazgo: Estimación, Planificación y Seguimiento de las tareas, además de la Organización, Dirección y Gestión de Recursos Humanos.
- **Desarrollo Técnico:** Comprende las actividades de **I**ngeniería del Software que crean el producto a lo largo de su ciclo de vida, incluyendo el Análisis, Diseño y la Codificación.
- **Sistema de Calidad:** Asegura que el producto final sea correcto y esté bien construido. Incluye:
    - **Validación:** Determina si el producto satisface las necesidades del usuario final (construyendo el producto correcto).

    - **Verificación:** Asegura que el producto se construye según los estándares y especificaciones técnicas (construyendo el producto correctamente).
    - **Pruebas:** Confirma el funcionamiento adecuado del código.
    - **Garantía de Calidad:** Conjunto de medidas preventivas para asegurar que se mantienen los niveles de calidad durante el proceso de construcción.
- **Sistema de Gestión de Configuración:** Aunque también se suele considerar a la Gestión de Configuración como una actividad de Garantía de Calidad.

El objetivo de las actividades de Gestión de Configuración del Software es:

Establecer y mantener la integridad de los productos generados durante un proyecto de desarrollo de software y a lo largo de todo el ciclo de vida del producto.
Evaluar y controlar los cambios sobre ellos, es decir, controlar la evolución del sistema software.
Facilitar la visibilidad sobre el producto.

¿Qué significa la integridad de un producto software?

Significa que el producto cumple las siguientes condiciones:

- Satisface las necesidades del usuario (cumple todos los requisitos del usuario, tanto los explícitos como los implícitos)
- Cumple los requisitos de rendimiento
- Se puede trazar su evolución desde que se concibió, y a través de todas las fases de su ciclo de vida.

Para conseguir estos objetivos, la Gestión de Configuración plantea la realización de ciertas actividades. La definición estándar de Gestión de Configuración del Software, tal y como aparece en el estándar de IEEE,incluye las siguientes actividades:

Identificación de la Configuración: Consiste en identificar la estructura del producto, sus componentes y el tipo de estos, y en hacerlos únicos y accesibles de alguna forma.

Control de Cambios en la Configuración: Consiste en controlar las versiones y entregas de un producto y los cambios que se producen en él a lo largo del ciclo de vida.

Generación de Informes de Estado: Consiste en informar acerca del estado de los componentes de un producto y de las solicitudes de cambio, recogiendo estadísticas acerca de la evolución del producto.

Auditoría de la Configuración: Consiste en validar la completitud de un producto y la consistencia entre sus componentes, asegurando que el producto es lo que el usuario quiere los sistemas que automatizan la gestión de configuración suelen incluir algunas funciones adicionales, lo que nos lleva a ampliar la definición estándar con las siguientes actividades:

Construcción: Consiste en gestionar la compilación y enlazado de los distintos componentes del producto software de una forma lo más eficiente posible.

Control del Trabajo en Equipo: Consiste en controlar las interacciones que se producen entre los múltiples desarrolladores de un producto, sobre todo cuando deben compartir ciertos componentes del producto.

Control de versiones: Consiste en mantener un registro histórico de las diferentes versiones por las que pasan los componentes de un producto, que permita la recuperación de cualquiera de ellas.

Gestión de Problemas: Consiste en llevar un seguimiento detallado de la evolución de todos los problemas, fallos o *bugs* que afectan al producto de software a lo largo de su ciclo de vida.

¿Por qué es compleja la Gestión de Configuración?

Las organizaciones grandes, o aquellas con aplicaciones de larga vida que manejan múltiples versiones simultáneas, enfrentan **necesidades de GCS muy grandes y complejas**. A pesar de esto, muchas empresas aún la realizan de manera mínima.

La complejidad se debe a factores clave:

Elevado Número de Componentes: A medida que el desarrollo avanza, la cantidad de componentes de software a controlar crece muy rápidamente.

El Factor Inevitable del Cambio: El problema principal surge porque el **cambio es constante e inevitable** a lo largo de todo el ciclo de vida del sistema (Ley de Bersoff). La GCS debe **asumir el cambio, no evitarlo**, entendiendo que la mayoría de los cambios están justificados por el conocimiento que se adquiere con el tiempo.

Interdependencia con Otros Problemas: La GCS se complica debido a la **falta de soluciones satisfactorias** para otros retos de la Ingeniería del Software. Por ejemplo:

- Sería más fácil si existieran mejores formas de describir la arquitectura, lo que permitiría que los cambios en el código se propagaran al resto del sistema de forma sencilla.
- La GCS está fuertemente ligada al **mantenimiento del software**; una GCS deficiente convierte el mantenimiento en una "verdadera pesadilla".

Tiene también una gran influencia sobre otros aspectos del desarrollo de software, como son:

Las metodologías: Las actividades de Gestión de Configuración deberán integrarse con la metodología de desarrollo que utilice la empresa. Su planificación dependerá de las fases que establezca la metodología, los productos que se generen, etc.

El entorno de desarrollo de software: cómo de fuertemente integrada está la gestión de configuraciones con el resto de las capacidades del entorno de desarrollo, y qué tipo de funciones de Gestión de Configuración permite.

El modelo de proceso software: La Gestión de Configuración del Software (GCS) tiene un impacto directo en cómo los desarrolladores realizan su trabajo, ya que **impone políticas y procedimientos** y supervisa su cumplimiento.

Madurez Organizacional: La GCS desempeña un papel crucial en el incremento del **nivel de madurez** del proceso software de una organización. Es un área clave requerida para que una organización alcance el **Nivel 2** del modelo **CMM (Capability Maturity Model)**.

La calidad del producto software: Puesto que contribuye a mantener la integridad del producto.

La organización: Puesto que por lo general se suele implantar una solución global, que afecta a toda la organización, y requiere del establecimiento de nuevos roles y responsabilidades que deberán integrarse en la organización del proyecto.

¿Qué es la Configuración del Software? ¿Y los Elementos de Configuración?

Configuración del Software: Es el **conjunto total** de toda la información y productos utilizados o generados en un proyecto como resultado del proceso de Ingeniería de Software.

Elemento de Configuración del Software (ECS): Es la unidad básica de trabajo para la GCS. Es cada componente individual de la configuración total. Un ECS debe ser una unidad que se pueda definir y controlar de forma separada.

Nota: El problema en GCS surge porque el número de ECS crece rápidamente a medida que avanza el proyecto, y todos están sujetos a la variable CAMBIO, lo cual aumenta la complejidad de mantener el control.

Se pueden considerar como Elementos de Configuración del Software los siguientes componentes, entre otros:

1. La especificación del sistema.
2. El plan del proyecto software.
3. La especificación de requisitos software.
4. Un prototipo, ejecutable o en papel.
5. El diseño preliminar.
6. El diseño detallado.
7. El código fuente.
8. Programas ejecutables.
9. El manual de usuario.
10. El manual de operación e instalación.
11. El plan de pruebas.
12. Los casos de prueba ejecutados y los resultados registrados.
13. Los estándares y procedimientos de ingeniería de software utilizados.
14. Los informes de problemas.
15. Las peticiones de mantenimiento.
16. Los productos hardware y software utilizados durante el desarrollo.
17. La documentación y manuales de los productos hardware y software utilizados durante el desarrollo.
18. Diseños de bases de datos.
19. Contenidos de bases de datos.

Criterios y Desafíos del Elemento de Configuración (ECS):

Criterio de Definición: Un Elemento de Configuración del Software (ECS) debe ser una **unidad controlable y definible por separado**.

Alcance de Control: La GCS no solo controla el software producido, sino a menudo los **productos hardware y software utilizados** para el desarrollo (ej.,

compiladores). Esto asegura que si se necesita rehacer una versión, las herramientas utilizadas sean las mismas, ya que una nueva versión de una herramienta podría generar resultados diferentes.

Descomposición: El software, dependiendo de su complejidad, puede ser descompuesto en **varios ECS**, aunque el sistema completo en sí mismo también se considera un ECS.

El Desafío del Crecimiento: El número de ECS **crece rápidamente** durante el desarrollo (un documento ECS da lugar a muchos otros). Aunque esto es esperable, la **confusión** real y el problema de gestión surgen cuando entra en juego la variable **CAMBIO**, que es la que multiplica la complejidad.

Líneas base

Una Línea Base es un **punto de referencia** formal en el desarrollo, marcado por la **aprobación** de uno o varios **Elementos de Configuración del Software (ECS)** mediante una **revisión técnica formal**.Se utiliza para **controlar los cambios** en el sistema sin impedir los cambios justificados.

Función: Un ECS que forma parte de una Línea Base se convierte en la **base firme para desarrollos posteriores**.

Control de Cambios:

- **Antes de la Línea Base:** Se permiten **cambios rápidos e informales**.
- **Después de la Línea Base:** Cualquier cambio requiere un **procedimiento formal** de control, evaluación y verificación.

Versiones, Revisiones, Variantes, Configuraciones Alternativas y Releases

La GCS requiere un control estricto sobre cómo evoluciona cada componente de software.

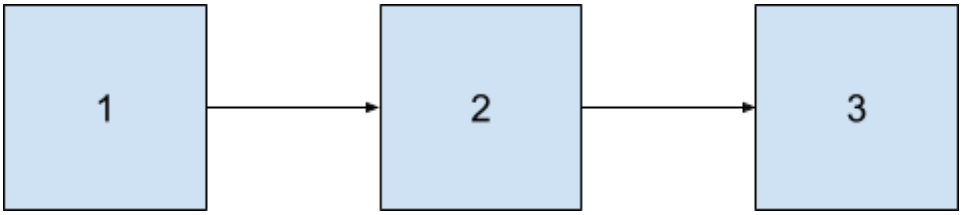
Versión: Es una **instancia** específica de un ECS en un **momento dado** del desarrollo, almacenada en un repositorio para poder ser recuperada, usada o modificada en cualquier momento.

Revisiones: Son las distintas **versiones temporales** que aparecen a medida que se avanza en el desarrollo de un elemento.

Cadena de Revisión: Es la forma más fácil de evolución, donde cada nueva revisión se crea modificando la más reciente, formando una **sucesión temporal**. La nueva revisión sustituye a la anterior, aunque todas deben ser accesibles.

Identificación: Cada revisión debe identificarse de manera **única**, siendo común el uso de un **esquema numérico sucesivo**.

Grafo de Evolución: Es una representación que muestra las diferentes revisiones de un elemento y las **relaciones de sucesión temporal** entre ellas.



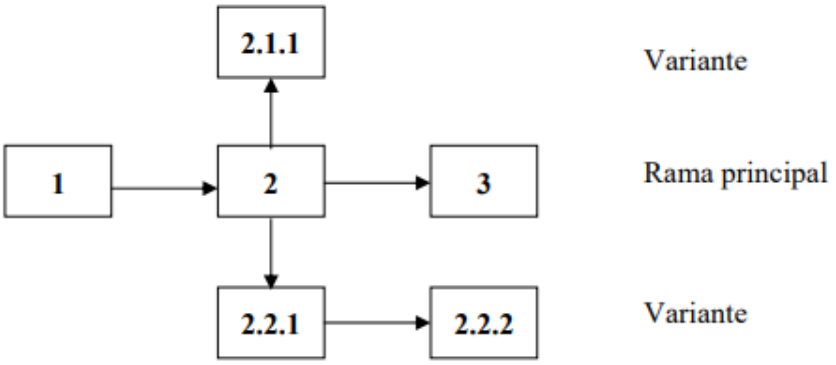
Las variantes de un Elemento de Configuración del Software (ECS) se dividen en temporales y permanentes, y una variante puede tener su propia sucesión de revisiones.

1. Variantes Temporales (Uso Transitorio): Las variantes temporales están destinadas a **mezclarse** con otra variante o ser **descartadas** en algún momento. **Propósito (Fusión Requerida):** Se crean para permitir que **varias personas trabajen en paralelo** sobre la misma versión de un objeto sin conflictos. Una vez terminadas las modificaciones, es **necesario mezclarlas** (fusionarlas) para integrar todos los cambios en la versión principal.

- ***Nota:*** Estas variantes deben **evitarse o mezclarse lo antes posible**, ya que su divergencia con el tiempo hace la fusión más difícil.

Propósito (Descarte):

- **Variantes Experimentales:** Se desarrollan en paralelo para **explorar distintas soluciones alternativas** cuando no se sabe cuál es la más adecuada. Las variantes descartadas son **abandonadas** (no se fusionan).
- **Variantes de Pruebas:** Se utilizan para **facilitar la realización de pruebas**, introduciendo elementos especiales (ej. trazar el flujo de control). Son variantes "de usar y tirar".



2. Variantes Permanentes (Coexistencia Constante): Las variantes permanentes aparecen para dar respuesta a **conjuntos distintos de requisitos** y **nunca se llegan a mezclar** entre sí.
Las variantes permanentes aparecen para dar respuesta a **conjuntos distintos de requisitos** y **nunca se llegan a mezclar** entre sí.

- **Variantes de Requisitos de Usuario:** Responden a diferentes necesidades de los usuarios (el caso más típico es el **idioma**).
- **Variantes de Plataforma:** Necesarias para que la aplicación funcione en distintas plataformas hardware o sistemas operativos.

Variantes de requisitos de usuario: Tenemos distintos usuarios con distintos requisitos. El caso más típico es el idioma usado por estos.

Variantes de plataforma: Debemos realizar una variante por cada sistema operativo o plataforma hardware sobre la que deseemos que funcione nuestra aplicación.

Configuraciones Alternativas: Debido a la existencia de **ramificaciones** (variantes), el sistema ya no tiene una única configuración. En su lugar, existe un conjunto de **CONFIGURACIONES ALTERNATIVAS**.

- **Propósito:** Cada configuración alternativa satisface las necesidades de un **entorno o usuario particular**.
- **Definición:** Se especifica seleccionando los **Elementos de Configuración del Software (ECS)** exactos y la **versión correcta** de cada uno.

Especificación de la Configuración (Selección de ECS): El proceso para seleccionar los ECS adecuados y construir una configuración alternativa se logra mediante:

1. **Atributos y Características Lógicas:** Se asocian **atributos** (ej., "color", "monocromo") a cada versión de un ECS. Las expresiones lógicas que

utilizan estos atributos para definir qué componentes incluir se llaman **CARACTERÍSTICAS LÓGICAS**.

2. **Especificación y Recuperación:** Se crea una especificación que describe el conjunto de atributos deseado, permitiendo a las herramientas **recuperar los ECS** adecuados para construir la configuración.
3. **Construcción Automática y Release:**
    a. **Herramientas de Construcción:** Herramientas como MAKE facilitan la construcción automática (recuperación, compilación y montaje) de una configuración concreta.
    b. **Funcionamiento de MAKE:** Utiliza un fichero (Makefile) para definir las relaciones entre componentes y revisa las fechas de modificación para compilar solo la cantidad mínima de componentes necesarios.
    c. **Limitación:** MAKE no es una herramienta de control de versiones y no puede especificar qué versión de un ECS usar, por lo que debe combinarse con una herramienta de control de versiones.
    d. **Release:** El término RELEASE se utiliza para designar la configuración final del sistema que se va a comercializar o entregar al cliente.

1.5. Almacenamiento de Versiones

1. Necesidad de Almacenar Versiones Anteriores

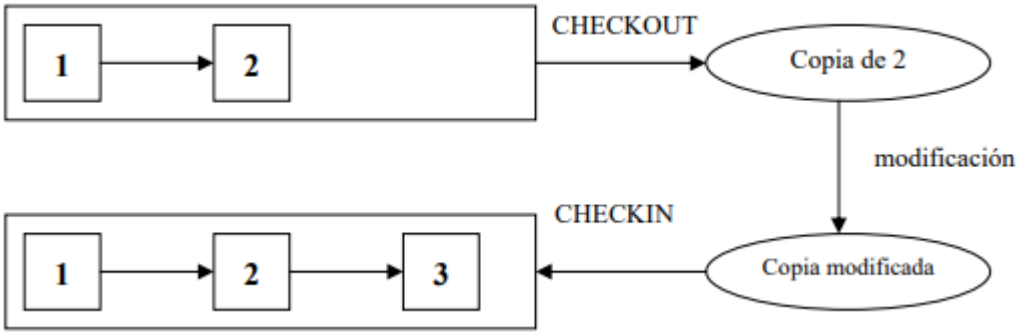
No se debe almacenar únicamente la última versión debido a varias necesidades:

- **Mantenimiento de Sistemas en Uso:** Versiones antiguas pueden seguir siendo parte de sistemas que aún están activos y deben ser **mantenidos**.
- **Punto de Seguridad:** Sirven como **puntos de reversión** (rollback) seguros para volver atrás si un cambio reciente no produce el resultado deseado.
- **Reutilización:** Permiten recuperar elementos que fueron **desechados** en versiones posteriores.

2. Creación y Congelamiento de Nuevas Versiones

- **Extremo Inviable:** Crear una nueva versión por **cada pequeña modificación** hace que el número de revisiones crezca de forma inmanejable.
- **Enfoque Práctico (Congelamiento):** Se permite al desarrollador modificar un ECS tantas veces como sea necesario bajo el mismo identificador hasta que la versión se considera estable y es **CONGELADA**.
    - Una vez congelada, la versión se **almacena** y ya **no puede ser modificada**.
    - Cualquier modificación posterior requiere la **creación de una nueva versión**

El modelo de trabajo sobre el que se basan la mayoría de las herramientas de gestión de revisiones es el de la figura:



Las herramientas de gestión de revisiones se basan en un modelo de **repositorio central** y directorios de trabajo locales:

CHECKOUT: Operación para extraer una copia de la última versión del objeto del repositorio a un directorio local.

CHECKIN: Operación para devolver la copia modificada al repositorio como una nueva versión sucesora.

Almacenamiento Eficiente de Versiones(Deltas): Para reducir el espacio, las herramientas usan un sistema de cambios incrementales.

DELTA: Es la secuencia de operaciones que, aplicadas a una revisión (R_1), resultan en la siguiente (R_2). Es la representación del **historial de cambios** necesario para reconstruir cualquier versión.

Tipo de Delta	Versión Base Almacenada	Función	Ventaja
Deltas Directos	La **primera** versión.	Cambios de versión vieja a nueva.	Menos cambios sobre la versión base.
Deltas Inversos	La **última** versión.	Cambios de versión nueva a anterior.	La **recuperación de la última versión es más rápida** (la que se usa con mayor frecuencia).

Almacenamiento: Los Deltas pueden guardarse en ficheros separados o integrarse en el mismo documento (**Deltas Mezclados**).

2. IDENTIFICACIÓN DE LA CONFIGURACIÓN

La Identificación de la Configuración es la primera de las cuatro tareas básicas de la Gestión de Configuración del Software (GCS).

Su objetivo principal es identificar y asignar nombres significativos y consistentes a todos los Elementos de Configuración del Software (ECS) del producto en cada fase de su desarrollo.

Esta tarea puede englobar las siguientes actividades:

- Establecimiento de una jerarquía preliminar del producto.
- Selección de los ECS.
- Definición de las relaciones en la configuración.
- Definición de un Esquema de Identificación.
- Definición y Establecimiento de Líneas Base.
- Definición y Establecimiento de Bibliotecas de Software.

2.1. Establecimiento de una jerarquía preliminar del producto software

Esta actividad busca obtener una **visión inicial de la estructura** y los elementos que compondrán el sistema.

- Propósito: Facilita actividades posteriores de GCS, como la selección de los ECS y la asignación de identificadores.
- Momento: Se realiza, opcionalmente, en los **primeros compases** del proyecto.

2.2. Selección de los Elementos de Configuración

La **selección de un número adecuado de ECS** es crucial para la manejabilidad y visibilidad del proyecto.

Escenario	Problema Principal	Consecuencias Adicionales
Demasiados ECS	Produce un número excesivamente alto de especificaciones y documentos,	El mantenimiento de cada EC es **costoso** y puede requerir: especificación, plan de pruebas, manuales, revisiones/aprobaciones del usuario, auditorías y contabilidad de estado separadas.

	volviéndose **inmanejable**.	
Muy pocos ECS	Causa una **falta de visibilidad** suficiente sobre el producto.	El impacto de los cambios será mayor, lo que obligará a repetir el proceso de reconstrucción y liberación de la versión con más frecuencia.

2.3. Definición de relaciones en la configuración

Se puede considerar que los ECS son objetos, y están conectados con otros ECS mediante relaciones. Esta información ayudará al personal de GCS a comprender dónde se sitúa un elemento con respecto al resto. Hay que tener en cuenta que el personal de GCS suele ser externo al equipo de desarrollo y necesita este tipo de ayudas para poder asomarse a los productos y el proceso de desarrollo.

Algunas de las relaciones que puede ser interesante mantener son las siguientes:

Equivalencia: Por ejemplo, cuando un determinado ECS que es un programa está almacenado en tres lugares diferentes (un disco maestro, una copia de seguridad en cinta y el diskette del programador), pero todas las copias corresponden al mismo programa.

Composición: Por ejemplo, el ECS “especificación de diseño” estará compuesto de otros ECS, como el “modelo de datos” o el “diseño del módulo N”, para cada uno de los módulos que componen el producto software.

Dependencia: Cualquier otro tipo de relaciones entre ECS, fundamentalmente en la documentación, y sobre todo para facilitar la trazabilidad de los requisitos. Así, por ejemplo, los objetos tienen dependencia del modelo de clases.

Derivación: A partir de qué se ha originado algo. Por ejemplo, el código objeto del código fuente, o una determinada traza de ejecución de un determinado caso de prueba con un determinado programa ejecutable.

Sucesión: En la historia de cambios sobre un elemento desde una revisión a otra. Puede ser muy útil definir un Grafo de Evolución para cada ECS. Este grafo describe la historia de cambios de un objeto y su transición de unas versiones a otras.

Variante: Variación sobre un determinado elemento, con la misma funcionalidad, pero que, por ejemplo, funciona más rápido.

La **Gestión de Configuración del Software (GCS)** modela los Elementos de Configuración del Software (**ECS**) como objetos conectados por **relaciones** para que el personal de GCS (a menudo externo al equipo de desarrollo) comprenda la estructura del sistema.

2.4. Definición de un Esquema de Identificación:

Esta actividad consiste en establecer un método para **etiquetar de forma unívoca** cada **Elemento de Configuración del Software (ECS)** y diferenciar sus versiones.

El esquema de identificación debe proporcionar al menos **diez campos de información clave** sobre el ECS:

1. Número o código del Elemento de Configuración del Software.
2. Nombre del ECS
3. Descripción del ECS
4. Autor/es del ECS
5. Fecha de creación
6. Identificación del proyecto al que pertenece el Elemento de Configuración del Software.
7. Identificación de la línea base a la que pertenece.
8. Identificación de la fase y subfase en la que se creó.
9. Tipo de Elemento de Configuración (documento, programa, elemento físico (cintas, discos, etc.),...).
10. Localización

Y crucialmente, campos para diferenciar sus **versiones**
11. Número de versión
12. Fecha de la versión

Tipos de Codificación: Se puede elegir entre dos sistemas para el código de identificación:

- **Codificación No Significativa:** Asigna números consecutivos (ej., 001, 002). Es simple, pero el código **no ofrece información** sobre el tipo de ECS o su función.
- **Codificación Significativa:** La estructura del código **incluye información codificada** sobre el ECS (ej., FASE-TIPO-NÚMERO). La ventaja es que se

puede identificar el ECS sin consultar un registro, pero si es demasiado complejo, se pierde la utilidad.

Almacenamiento: Los datos de identificación se mantienen idealmente en una **base de datos automatizada**. Las herramientas de GCS suelen soportar bien la identificación de componentes de programa, pero menos la de documentos o dispositivos físicos.

2.5. Definición y Establecimiento de líneas base

Las Líneas Base son el concepto clave para la identificación y se establecen en hitos (milestones) previamente especificados del proceso de desarrollo.

Propósitos del Establecimiento de Líneas Base

Las Líneas Base se establecen en hitos clave del ciclo de vida con dos objetivos principales:

1. Identificar los resultados y productos finales de la fase.
2. Asegurar que la fase ha sido completada y aprobada.

Identificar los resultados de las tareas realizadas durante la fase.
Asegurar que se ha completado la fase. Aunque se pueden definir las líneas base con cualquier nivel de detalle, las líneas base más comunes son:

Línea Base Funcional: Que se establece al finalizar la fase de análisis y especificación de los requisitos del sistema, y comprende todos aquellos documentos en los que se define el problema a resolver, los costes del proyecto, el plan de tiempos, y los diferentes requisitos funcionales, de interoperabilidad y de interfaz del sistema.

Línea Base de Distribución o Asignación de funciones: Que se establece al finalizar la fase de análisis y especificación de requisitos software, y comprende toda la documentación que gobernará el desarrollo de cada uno de los componentes software que se han identificado en la especificación del sistema, y la asignación o reparto de las diferentes funciones entre los distintos componentes del sistema.

Línea Base de Diseño Preliminar: Que se establece al finalizar la fase de diseño preliminar. Comprende todos aquellos documentos en los que se define la arquitectura del producto software, así como el Plan de Pruebas.

Línea Base de Diseño: que se establece al finalizar la fase de diseño detallado. Comprende todos aquellos documentos que contienen el diseño detallado del

software y el plan de implementación, y también la descripción de los casos de prueba.

Línea Base de Producto: que se establece al finalizar la fase de pruebas. Comprende los programas creados y todos aquellos documentos que contienen la información relativa a las pruebas realizadas.

Línea Base de Operación: que se establece al finalizar la fase de implantación. Comprende los manuales de usuario, guías de operación y mantenimiento, manuales de formación, etc.
Hay autores que sólo consideran las líneas base Funcional, de Asignación, y de Producto, más una Línea Base de Desarrollo, que va evolucionando a lo largo del desarrollo, y que incluiría la Línea Base de Diseño Preliminar y la Línea Base de Diseño.

Una línea base se puede establecer de dos formas:

- **FÍSICAMENTE:** **Etiquetando** los ECS y almacenándolos en una Biblioteca de Proyecto.
- **LÓGICAMENTE:** Publicando un documento que identifica el estado actual de la configuración en ese punto del proceso.

2.6. Definición y Establecimiento de bibliotecas de software

Una **Biblioteca de Software** es una colección controlada de software y/o documentación cuyo objetivo es apoyar el desarrollo y mantenimiento. Su propósito es facilitar la GCS, especialmente en el control de cambios.

Tipos de Bibliotecas y Control Aplicado

Tipo de Biblioteca	Propósito y Contenido	Nivel de Control de Cambios
Biblioteca de Trabajo	Área de trabajo para analistas y programadores (codificación y pruebas unitarias).	Informal.
Biblioteca de Integración	Fuente de ECS para su integración en componentes de nivel superior del sistema.	

Biblioteca de Soporte al Proyecto	Almacena ECS aprobados y transferidos desde las bibliotecas de trabajo/integración.	Interno y Semiformal.
Biblioteca Maestra	Almacena las Releases liberadas para entrega al cliente o comercialización.	**Formal y Estricto.** Fuertes restricciones de acceso para escritura.
Repositorio de Software	Almacena copias de seguridad.	Adecuada identificación y catalogación (central y común a todos los proyectos).
Biblioteca de Backup	Almacena copias de seguridad.	No sujeta a GCS.

3. CONTROL DE CAMBIOS EN LA CONFIGURACIÓN

El **Control de Cambios en la Configuración** es la **actividad más importante** de la GCS. Su objetivo es proporcionar un mecanismo **riguroso** para controlar la inevitable evolución del sistema, combinando procedimientos humanos (Comité) y herramientas automáticas.

Tipos de Cambios

Corrección de un defecto: (Usado típicamente por clientes).

Mejora del sistema: (Usado típicamente por programadores).

Niveles de Control de Cambios:

Nivel de Control	Momento de Aplicación	Responsabilidad y Aprobación
Semi-Formal (Nivel Proyecto)	Una vez que el ECS es una Línea Base.	Aprobación del **Director del Proyecto** (cambios locales) o del **Comité de Control de Cambios (CCC)** (cambios con impacto).
Formal	Una vez que el producto se **comercializa** (ECS transferidos a la Biblioteca Maestra).	ATodo cambio debe ser aprobado por el **Comité de Control de Cambios (CCC)**.

El Comité de Control de Cambios (CCC): El CCC es la figura o grupo encargado de tomar las **decisiones finales** sobre el estado y la prioridad de las peticiones de cambio. Su función es tener una visión general para evaluar el **impacto** del cambio (coste, rendimiento, fiabilidad, etc.).

Todos los miembros de un proyecto: Solicitan, realizan e informan sobre los cambios.

El jefe de proyecto: Asegura que los procedimientos se sigan, evalúa el impacto y el coste.

El bibliotecario: Controla los cambios sobre las versiones adecuadas y transfiere los elementos entre las bibliotecas (ej., de Soporte a Trabajo y viceversa, sin tocar la Biblioteca Maestra hasta que el cambio se prueba).

Formalización de Políticas: Al inicio de cada proyecto, es obligatorio establecer **Políticas, Estándares y Procedimientos** que definan el "qué", "cuándo", "cómo" y "quién" del proceso de GCS (ej., responsabilidades, contenido de las líneas base y criterios de valoración de solicitudes).

3.1. El proceso de control de cambios

El proceso formal de control de cambios es una secuencia de pasos típica para modificar una Línea Base aprobada:

1. Iniciación del Cambio: se presenta una solicitud de cambio, que puede venir provocada por un problema que se ha detectado o por un cambio en los requisitos.

2. Clasificación y registro de la solicitud de cambio.

3. Aprobación o rechazo inicial de la solicitud de cambio: Decisión tomada por el **CCC**.

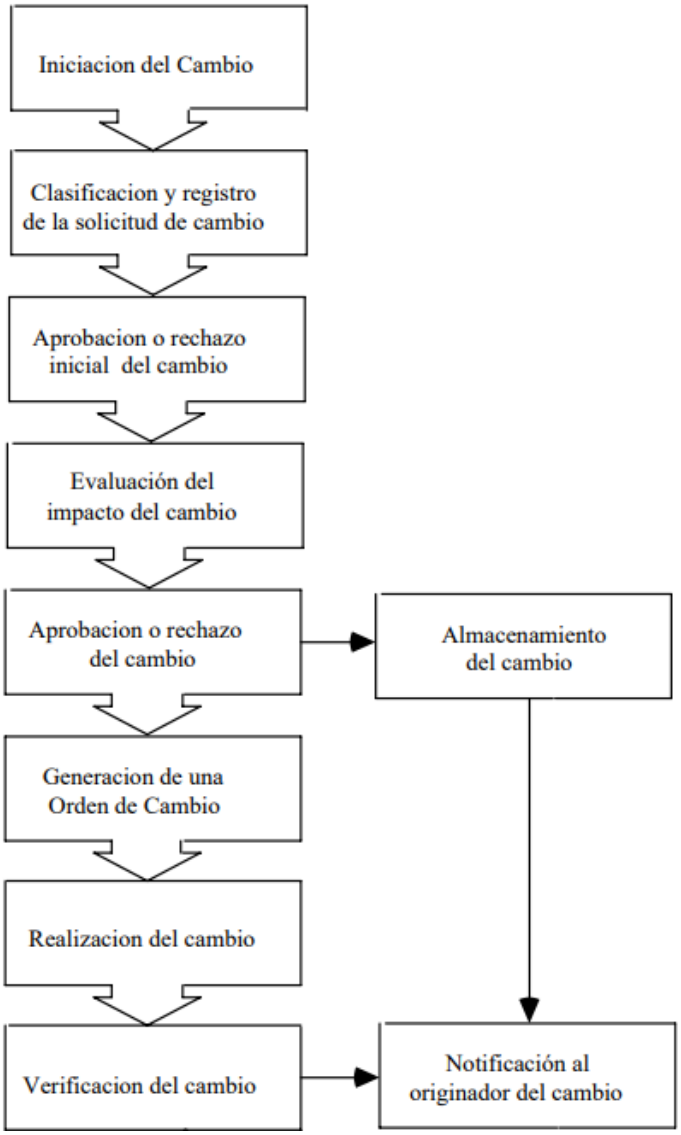
4. Evaluación de la solicitud de cambio: Si se aprueba, se calcula el **esfuerzo técnico**, el **impacto global** y el **coste** del cambio. El resultado es un **Informe de Cambio**.

5.Orden de Cambio: El CCC genera una **Orden de Cambio** (Orden de Cambio de Ingeniería), que asigna la tarea al ingeniero. El objeto a cambiar se da de baja en la Biblioteca de Soporte al Proyecto.

6. Realización y Seguimiento: El ingeniero lleva a cabo el cambio bajo control.

7. Certificación y Revisión: Se verifica que el cambio se realizó correctamente, corrigiendo el problema o satisfaciendo el requisito modificado. El objeto se devuelve a la Biblioteca de Soporte al Proyecto.

8. Notificación: Se informa al originador del cambio del resultado.



Proceso de Control de Cambios Semi-Formal:En un proceso de control de cambios **semi-formal** (típico a nivel de proyecto), se puede simplificar el proceso eliminando parte de la documentación estricta:

- Se puede suprimir la necesidad de generar una **solicitud, informe u orden de cambio**.
- Sin embargo, la **evaluación del impacto** del cambio y su **seguimiento** posterior siempre deben realizarse.

Obligación de Definición: Al comienzo de cada proyecto, es necesario establecer con precisión el proceso de control de cambios a utilizar, lo que incluye definir los mecanismos para:

- Solicitar y aprobar/rechazar cambios.
- Analizar y evaluar el impacto de las solicitudes.
- Controlar la realización de los cambios aprobados.

Mecanismos de Control de la Biblioteca (Seguridad y Coordinación): Los procesos de alta y baja (check-in/check-out) de la Biblioteca del Proyecto implementan dos controles esenciales:

Mecanismo de Control	Objetivo	Función Principal
Control de Acceso	**Seguridad.** Define los **derechos** que tienen los diferentes miembros para acceder y modificar Elementos de Configuración del Software (ECS) concretos.	Asegura que solo el personal autorizado pueda modificar un ECS, especialmente cuando se da de baja para realizar un cambio.
Control de Sincronización	**Coordinación.** Ayuda a asegurar que los **cambios en paralelo** realizados por diferentes personas o equipos no se sobreescriban entre sí.	**Bloquea** el objeto en el repositorio (Biblioteca de Soporte) al realizar un *check-out*, impidiendo otras actualizaciones hasta que se inserte la nueva versión (*check-in*).

3.2. Mecanismo para solicitar cambios

El proceso inicia con un formulario (**Solicitud de Cambio**) que debe ser **simple** y contener información clave como: **por qué** se requiere el cambio, **qué** se debe cambiar, **quién** lo solicita y **otros ECS afectados**.

- Por qué se necesita el cambio (el elemento no funciona tal y como debe hacerlo, le falta alguna característica, etc.)
- Qué hay que cambiar
- Quién lo solicita
- Descripción del problema lo suficientemente detallada como para que se pueda recomendar una solución.
- Otros ECS que se pueden ver afectados por el cambio.
- Otros cambios con los que está relacionado.
- Aprobación del cambio.
- Cómo se solucionó el problema, etc.

Informes de Incidencias (Informes de Problemas): Se usan para reportar un problema o deficiencia del sistema (ej., durante pruebas o uso). Recogen datos específicos del incidente (fecha, hora, cómo duplicarlo, efectos).

A los formularios que se usan para informar de problemas se les suele llamar Informes de Incidencias o Informes de Problemas. Recogen información adicional sobre el incidente que ha desvelado la existencia de un problema:

Acciones Recomendadas tras Incidencia:

No requiere acción: Si es un malentendido o ya se está corrigiendo.

Solicitud de Cambio: Si el problema requiere una **mejora en el diseño** del sistema.

Notificación de Cambio: Si el problema se debe a una **mala implementación** que debe corregirse inmediatamente (puede responder a varios Informes de Incidencias).

3.3. Mecanismo para aprobar o rechazar las solicitudes de cambio

Algunos criterios que se pueden tener en cuenta para tomar la decisión de aprobar o rechazar las solicitudes de cambio son los siguientes:

- Valor del cambio para el proyecto/organización
- Retorno de la inversión
- Tamaño
- Complejidad
- Impacto sobre el rendimiento del producto (uso de memoria y CPU)
- Recursos disponibles para efectuar el cambio (humanos y materiales)
- Relación con otros cambios ya aprobados y en progreso
- Tiempo estimado para completar el cambio
- Relación con las políticas de la empresa (satisfacción del cliente, competitividad, etc.)
- Existencia de alternativas, etc.

3.4. Seguimiento de los cambios. Gestión de Problemas

La **Gestión de Problemas** es una actividad **complementaria** al Control de Cambios, enfocada en gestionar la evolución de los problemas detectados (tanto en pruebas como por usuarios).

Conlleva tareas como:

Tareas: Admisión, registro, asignación de responsables, **monitorización del estado**, asociación de información en una **Base de Datos de Problemas** y

generación de informes estadísticos (ej., correlaciones entre problemas y componentes).

Datos Clave: La base de datos almacena información vital como: **severidad, urgencia, causa** (ej., error de análisis, falta de experiencia), solución, módulos afectados y las fechas clave (notificación, resolución).

4. GENERACIÓN DE INFORMES DE ESTADO

La **Generación de Informes de Estado** (también llamada **Contabilidad de Estado**) es la tercera tarea básica de la GCS. Su objetivo es mantener a todos los participantes informados del **estado de la configuración y su evolución**, respondiendo a las preguntas: **"¿Qué ocurrió?"** y **"¿Cuándo ocurrió?"**

Importancia y Productos

- **Importancia:** Mantiene la **continuidad del proyecto**, evita la **duplicidad de trabajo** y permite aprender de aciertos y errores.
- **Actividades:** Captura de la información (que proviene de las otras tres tareas de GCS), **almacenamiento en una base de datos automatizada**, y generación de informes.
- **Productos:** **Registros** e **Informes**.

4.1. Registros

La GCS requiere capturar y almacenar la mayor cantidad de información posible en una base de datos para análisis *post-mortem* y estimaciones futuras. Los registros más comunes incluyen

a) Registro de elementos de configuración: Conteniendo toda la información relativa a los diferentes elementos de configuración.

b) Registro de Líneas base: Información sobre su composición y fecha de establecimiento.

c) Registro de Solicitudes de cambios: Detalle de las peticiones (código, solicitante, descripción, resolución) y la historia de los cambios aprobados (coste, esfuerzo, impacto, fechas de implementación/cierre).

d) Registro de Cambios: Contiene la información recogida en la Orden e Informe de Cambio, detallando la **evaluación** (coste, esfuerzo, soluciones), el **plan de implementación**, las restricciones y el **impacto sobre la configuración** (Líneas

Base y ECS afectados). Incluye una **historia completa** del cambio (fechas de solicitud, aprobación, implementación y cierre).

e) Registro de Incidencias: Información detallada sobre el **incidente** (tipo, fecha, historia) y el **resultado de la incidencia**, incluyendo la disposición del CCC y la referencia a la Solicitud o Notificación de Cambio generada (si aplica).

f) Registro de Modificaciones del Código: Detalle del cambio realizado en el código, incluyendo el número de identificación de la modificación, a qué solicitud/notificación corresponde, los **módulos afectados**, las **versiones modificadas** y las fechas de inicio/terminación.

Registros de Modificaciones Específicas: Información detallada de las modificaciones específicas en código, bases de datos y documentación (qué se cambió, quién lo hizo y a qué solicitud corresponde).

Registro de Releases y Variantes: Descripción de la **composición y estado** de las versiones liberadas, incluyendo los ECS que la componen, su versión y las **diferencias con la *release* anterior**.

Registro de Instalaciones: Seguimiento de dónde se ha instalado cada *release* del producto (lugar, fecha de instalación, *release* instalada).

Actas del CCC: Registro formal de las **decisiones y acciones** tomadas por el Comité, incluyendo los resultados de las **Auditorías** (deficiencias detectadas, plan de resolución y recomendaciones).

4.2. Informes

En cuanto a los informes, podemos distinguir dos tipos:

- Planificados
- Bajo demanda

Algunos ejemplos de los tipos de informes que se pueden generar son:

a) Informe de estado de los cambios: Es un resumen del estado en que se encuentran todas las solicitudes de cambio registradas durante un determinado período de tiempo.

b) Inventario de elementos de configuración: para ofrecer visibilidad sobre el contenido de las bibliotecas de proyecto.

c) Informe de incidencias: Es un resumen del estado en que se encuentran todas las incidencias originadas durante un determinado período de tiempo y las acciones a las que han dado lugar.

d) Informe de modificaciones: Es un resumen de las modificaciones que se han efectuado en el producto software durante un determinado período de tiempo.

e) Informe de diferencias entre versiones: Resumen los cambios y las diferencias exactas entre versiones sucesivas de un ECS en un período..

5. AUDITORÍA DE LA CONFIGURACIÓN

La Auditoría es una **verificación independiente** para evaluar la conformidad del producto respecto a especificaciones, estándares y requisitos. Es la forma de **comprobar que el producto es lo que pretende ser**.

Rol y Costo: A menudo se realiza en la intersección de GCS y Garantía de Calidad. Es la actividad de GCS **más costosa** y debe ser realizada por **personal experimentado ajeno al equipo de desarrollo** para mantener la objetividad.

Tipos de Actividades de Revisión:

Revisiones de fase: Se realizan al finalizar cada fase (donde se establecen Líneas Base) para **descubrir problemas** (no solo obvios).

Revisiones de cambios: Se realizan tras la implementación de un cambio aprobado para verificar que este se hizo **correctamente**.

Auditorías: Se realizan al final del desarrollo para examinar el producto en su conjunto.

Las revisiones se deben realizar de forma continua, durante todo el proceso de desarrollo, y no sólo al finalizar éste, cuando los problemas ya no tienen solución.

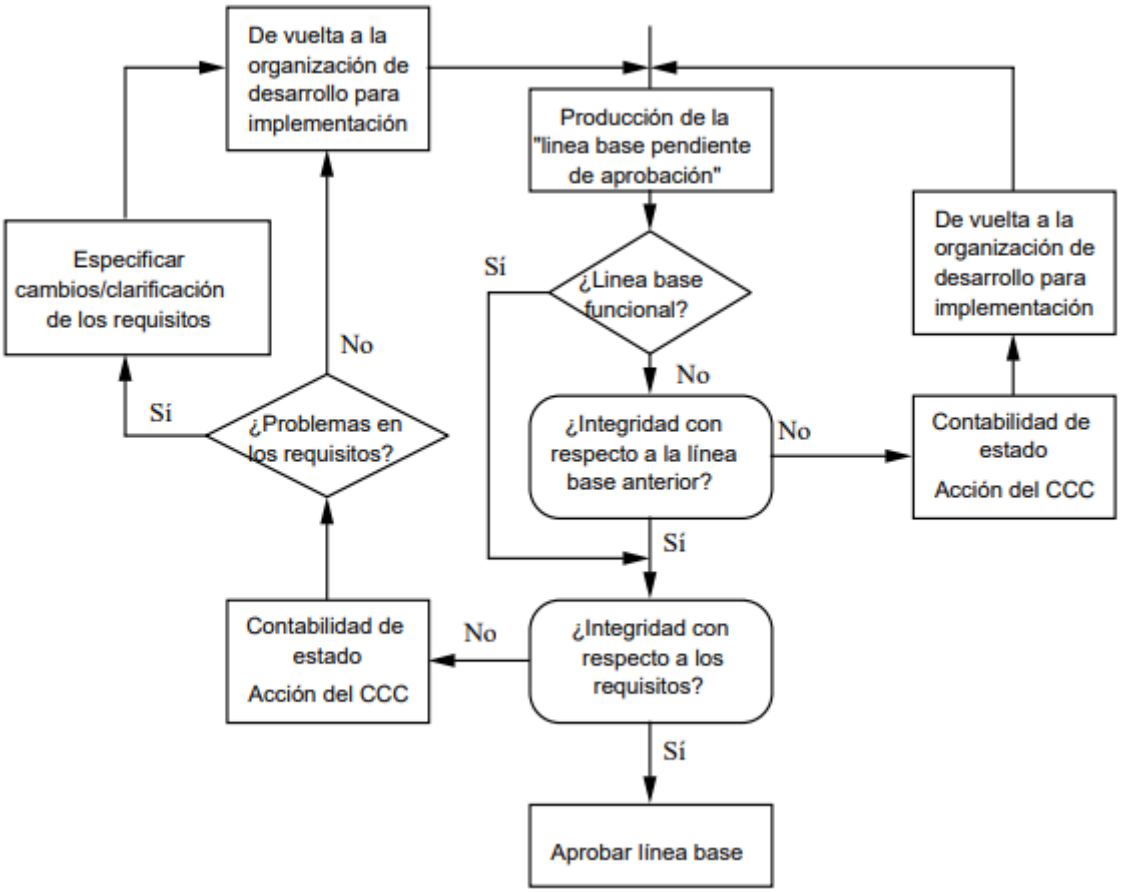
Toda revisión implica tres funciones esenciales:

Verificar: Comprobar que la configuración actual se **corresponde** con la que era en fases anteriores (respecto a la Línea Base precedente).

Validar: Confirmar que la configuración actual **satisface la función esperada** (respecto a los requisitos del sistema).

Valorar: Decidir si, tras la verificación y validación, la Línea Base es **aceptable**.

Flujo de Aprobación de la Línea Base: El papel de las revisiones es guiar la aprobación. Los productos se agrupan en una **"línea base pendiente de aprobación"** en un hito. Tras la **revisión de fase**, si hay deficiencias, se generan Informes de Problemas para que el **CCC recomiende cambios**. Después de implementar y verificar esos cambios (revisión de cambios), si no hay deficiencias, la línea base se declara **"aceptable"** y el CCC la aprueba.



Proceso y Tipos de Auditoría

Tipo de Auditoría	Objetivo	Momento
Auditoría Funcional	Comprobar que se completaron todos los *tests* y que el ECS **satisface los requisitos** impuestos.	Se realiza antes de la Auditoría Física.
Auditoría Física	**Verificar la adecuación y precisión de la documentación** (*Líneas Base de diseño y producto*) para asegurar que representan el software codificado y probado.	Se realiza inmediatamente después de la Auditoría Funcional, y tras ella **se establece la Línea Base de Producto**.
Revisión Formal de	Certificar que el ECS se comporta	Posterior a la entrega/instalación.

Certificación	correctamente **en su entorno operativo**.	